DIPLOMA: INFORMATION TECHNOLOGY

Subject: Business Analysis 3 Module 2/ Business Analysis 3.2

Subject code: AIBUY3A

Assessment: Project

Date issued: 10 August 2026

Lecturer: Dr Sibanda

Mr. Matsela

Mr. Matshego

Examiner: Mr Matsela

External Moderator: Mr XP Piyose

Total Marks: 100

Due date: 02 November 2026

Name and surname	Student number
TSC Mathelela	224061569
KT Mudau	225126079
S Mahlangu	224061569
M mapheto	223311053
KS Dlamini	225131099
T Ramohlakoana	240175816

Contents

Retail-X-AI

Introduction

1.1 Project Background and Relevance to the Theme

1.2 Problem Definition

1.5 Proposed AI Solution

1.6 Project Scope Overview

2.1 Business Background

2.3 Stakeholder Identification and Analysis

2.4 Business Case and Justification

2.5 Expected Business Benefits

1. Improved Demand Forecasting

2. Reduced Stockouts

3. Reduced Inventory Losses

4. Better Restocking Decisions

5. Improved Business Visibility

6. Improved Decision-Making

7. Improved Product Availability

2.6 Project Scope

2.6.1 In-Scope Activities

2.6.2 Out-of-Scope Activities

2.6.3 Scope Limitations

CHAPTER 3: REQUIREMENTS ANALYSIS

3.1 Introduction

3.2 Functional Requirements

3.4 Business Rules

3.5 User Stories

3.6 Acceptance Criteria

CHAPTER 4: SYSTEM ANALYSIS AND DESIGN

4.1 System Overview

4.2 System Actors

4.9 Chapter Summary

CHAPTER 5: DATASET AND DATA PREPARATION

5.1 Introduction

5.2 Dataset Source

5.3 Dataset Structure

5.4 Dataset Quality Assessment

5.5 Data Cleaning and Preprocessing

1. Data type conversion

2. Missing value assessment

3. Duplicate detection

4. Required field validation

5. Negative value assessment

6. Schema validation

5.6 Feature Engineering

5.6.1 Calendar Features

5.6.2 Categorical Encoding

5.6.3 Historical Demand Features

5.6.4 Excluded Features

5.7 Training, Validation and Testing Data

5.8 Data Preparation Pipeline

Figure 5.1: Retail-X-AI Data Preparation Pipeline

5.9 Dataset Limitations

5.10 Summary

AI DEVELOPMENT AND IMPLEMENTATION

6.1 Introduction

6.2 Development Environment and Technologies

6.3 Demand Forecasting Using Ridge Regression

6.3.1 Feature Preparation

6.3.2 Model Selection

6.3.3 Model Training

6.3.4 Model Comparison Results

6.3.5 Final Model Evaluation

6.4 Time Series Analysis

6.5 Inventory Analysis and Stockout Risk Detection

6.5.1 Safety Stock Calculation

6.5.2 Reorder Point Calculation

6.5.3 Target Stock Calculation

6.5.4 Stock Risk Classification

6.6 Smart Restocking Recommendations

6.7 Natural Language Processing and AI Chatbot

6.8 Techniques Used to Support Forecasting Accuracy

6.9 Integration of AI Components

6.10 Limitations of the AI Implementation

6.11 Chapter Summary

APPLICATION AND DATABASE IMPLEMENTATION

7.1 Introduction

7.2 Application Development

7.2.1 Application Overview

7.2.2 Application Navigation and Features

1. Dashboard

2. Sales Analysis

3. AI Forecasting

4. Smart Restocking

5. Inventory Alerts

6. AI Chatbot

7. Model Performance

7.3 Application Structure

7.4 Database Implementation

7.4.1 Database Overview

7.4.2 Database Technology

7.4.3 Database Table Structure

7.4.4 Database Import Process

7.4.5 Database Relationships and Logical Design

7.4.6 Database Verification

7.5 Integration Between the Application and Database

7.6 Application Limitations

7.7 Summary

TESTING AND RESULTS

8.1 Introduction

8.2 Functional Testing

8.3 AI Forecasting and Restocking Validation

8.3.1 Forecasting Results

8.3.2 Restocking and Stockout Results

8.4 Integration Testing

8.5 Performance Testing

8.6 Security Testing

8.7 Summary of Testing Results

8.8 Testing Limitations

CONCLUSION AND RECOMMENDATIONS

9.1 Project Conclusion

9.2 Recommendations

9.3 Future Improvements

9.4 Final Conclusion

Introduction

1.1 Project Background and Relevance to the Theme

Retail-X-AI is an AI-powered inventory management and demand forecasting solution designed for local spaza shops and small grocery businesses. It addresses challenges associated with manual stock management, inaccurate demand estimation, product shortages and unnecessary stock accumulation. The project aligns with the theme "An AI Solution for Industries" by applying Artificial Intelligence and Python-based technologies to improve inventory management within the retail industry. The proposed solution uses sales and inventory data to analyse purchasing patterns, forecast future

product demand, identify possible stock shortages and recommend suitable restocking quantities. By helping small retailers make better stock-related decisions, Retail-X-AI aims to reduce business losses, improve product availability and support the sustainability of local businesses that provide essential goods to surrounding communities.

1.2 Problem Definition

Small retail businesses, particularly spaza shops and local grocery stores, often struggle to manage inventory effectively. Many rely on manual stock recording and personal judgement to decide what to buy and when to restock. This can lead to inaccurate stock records, frequent shortages of essential products, overstocking, and financial losses from unsold or expired products.

Inaccurate demand forecasting makes it difficult for shop owners to understand purchasing patterns and prepare for changes in customer demand. These challenges can also affect surrounding communities when customers cannot access essential products from nearby shops.

Retail-X-AI aims to address this problem by using Artificial Intelligence and Python to analyse historical sales and inventory information. The system is intended to forecast product demand, monitor stock levels, identify potential shortages and recommend appropriate restocking quantities. This can help retailers make informed inventory decisions, reduce unnecessary losses and maintain the availability of essential products. For local communities, improved inventory management can support more reliable access to everyday goods while contributing to the sustainability of small businesses.

1.3 Main Project Objective

The main objective of Retail-X-AI is to develop an AI-powered inventory management and decision-support system using Python that analyses historical retail sales and

inventory information, forecasts product demand, identifies potential stockout risks and provides restocking recommendations through an accessible dashboard and chatbot.

1.4 Specific Project Objectives

The specific objectives of the project are to:

1. Analyse historical retail sales and inventory information to identify product performance and purchasing patterns.
2. Prepare and process retail datasets for use in machine learning and analytical activities.
3. Develop a machine learning forecasting solution to estimate product demand using historical data.
4. Evaluate forecasting performance using appropriate model evaluation metrics.
5. Identify potential stockout risks using available inventory and demand information.
6. Generate advisory restocking recommendations based on historical inventory and forecast information.
7. Develop a dashboard that presents sales analysis, demand forecasts, inventory alerts and restocking information.
8. Develop a chatbot that allows users to retrieve supported sales, inventory, forecasting and restocking information through natural-language questions.
9. Test the application's functionality and evaluate the reliability and limitations of the developed solution.

1.5 Proposed AI Solution

Retail-X-AI uses Python-based data processing and machine learning to support inventory management decisions. The project uses historical retail information, including sales quantities, inventory levels, product identifiers, dates, and other relevant factors. The main documented forecasting pipeline uses Ridge Regression to estimate product demand from prepared features. Forecast outputs are then used alongside historical inventory information to produce advisory stockout risk results and restocking recommendations.

The application presents these results through a Streamlit dashboard containing sales analysis, AI forecasting, smart restocking and inventory alert pages. It also includes a chatbot that uses predefined intent recognition and read-only data queries to respond to supported questions about the available retail information.

The solution currently operates using historical datasets and saved model outputs. It is therefore intended as a decision-support system rather than a live inventory management or automatic purchasing platform. Interpret its recommendations in light of the quality and limitations of the available data.

1.6 Project Scope Overview

The project focuses on historical retail data analysis, demand forecasting, inventory risk identification, restocking recommendations, dashboard development and chatbot functionality. It also includes database design, application development, testing and documentation.

The current implementation does not establish a live connection to a shop's point-of-sale system, suppliers or real-time inventory records. The selected dataset also does not support product expiry monitoring because it does not include product expiry-date information. These limitations are considered when evaluating the developed system.

2.1 Business Background

Retail-X-AI is designed around a fictional small retail business operating as a local spaza shop. The business represents a typical small-scale retailer that sells everyday consumer products such as groceries, beverages, household necessities and other essential items to customers within its surrounding community.

The fictional business operates in the retail industry and depends on maintaining sufficient stock to meet customer demand. Effective inventory management matters because the business needs to identify which products sell frequently, which move slowly, how much stock is available, and when to purchase more.

Without an effective inventory management and demand forecasting solution, the business may face challenges such as stock shortages, excess inventory, inaccurate purchasing decisions, and limited understanding of sales trends. These challenges can affect profitability, customer satisfaction and the availability of essential products.

Retail-X-AI is proposed as a technology-based solution to support the fictional business. It uses historical retail data, Python-based machine learning and data analysis to provide demand forecasts, inventory alerts and restocking recommendations. The system also provides a Streamlit dashboard and chatbot that let users access and interpret available business information.

The fictional spaza shop serves as the case study for analysing the business problem, identifying stakeholder requirements and demonstrating how Artificial Intelligence can support inventory management within small retail businesses.

2.2 Existing Business Challenges

The following challenges will form the basis of the business analysis:

Challenge	Possible effect on the business
Stock shortages	Lost sales and dissatisfied customers
Overstocking	Money tied up in unsold products
Poor demand estimation	Incorrect purchasing decisions
Slow-moving products	Products remain in inventory for long periods
Limited sales visibility	Difficulty identifying sales patterns
Manual inventory management	Greater possibility of inaccurate records

2.3 Stakeholder Identification and Analysis

The Retail-X-AI system involves several stakeholders with different responsibilities, interests, and expectations. Identifying these stakeholders helps ensure that the system addresses the needs of the fictional spaza shop and its surrounding business environment.

The main stakeholders are identified as follows:

Stakeholder	Role and responsibilities	Interest in the system

Shop Owner	Oversees the business, makes purchasing decisions and monitors overall business performance.	Wants to improve inventory management, reduce losses and make informed business decisions.
Shop Manager	Supervises daily shop operations and monitors stock availability.	Wants to identify low-stock products, review forecasts and manage restocking activities.
Shop Employees	Assist with daily operations and may be responsible for recording sales and updating stock information.	Need a system that is easy to use and helps them access accurate inventory information.
Customers	Purchase products from the spaza shop and depend on the availability of essential goods.	Benefit from improved product availability and fewer stock shortages.
Suppliers	Provide products to the shop and fulfil stock orders.	Benefit from clearer purchasing requirements and more organised restocking requests.
System Administrator	Responsible for maintaining the technical operation and configuration of the system.	Wants the application to remain functional, reliable and properly maintained.

Stakeholder Importance

The shop owner and shop manager are the primary stakeholders because they are the main decision-makers who would use the system's information to manage inventory and make purchasing decisions.

Shop employees are secondary users who may help record and access business information. Customers and suppliers are external stakeholders who benefit indirectly from improved inventory management. The system administrator supports the application's technical maintenance and operation.

Understanding these stakeholders helps guide the development of system requirements, user interactions and application functionality.

2.4 Business Case and Justification

The proposed Retail-X-AI system aims to address inventory management challenges faced by small retail businesses. Traditional inventory management methods may provide limited insight into historical sales patterns, future product demand and appropriate stock levels. This can make it difficult for shop owners and managers to make accurate purchasing decisions.

Retail-X-AI provides a data-driven approach by using historical retail information and machine learning techniques to analyse product demand. The system generates demand forecasts, identifies potential stockout risks and provides restocking recommendations to support inventory-related decisions.

Artificial Intelligence is appropriate for this project because machine learning techniques can identify patterns in historical sales data and use those patterns to estimate future demand. Instead of relying entirely on manual judgement, the business can use the information generated by the system to support its purchasing and inventory decisions.

The project also provides a central dashboard where users can view sales information, forecasting results, inventory alerts and restocking recommendations. The chatbot provides an additional way for users to retrieve supported business information.

The proposed business value includes better inventory visibility, improved purchasing decisions, reduced risk of product shortages and more efficient management of available stock. The project therefore shows how AI can support small businesses in the retail industry.

2.5 Expected Business Benefits

Retail-X-AI is expected to provide the following business benefits:

1. Improved Demand Forecasting

The system analyses historical sales information to estimate future product demand. This can help the shop owner make better purchasing decisions and prepare for changes in customer demand.

2. Reduced Stockouts

Inventory alerts and demand forecasts can help identify products that may run out of stock. The project proposes reducing stockouts by 30%. This is an expected business target and has not been achieved.

3. Reduced Inventory Losses

The system provides information to help the business avoid unnecessary stock accumulation and make more appropriate purchasing decisions. The proposed target is to reduce inventory losses by 20% to 25%. This target has not yet been verified through actual business operations.

4. Better Restocking Decisions

Restocking recommendations guide which products may need additional stock and the quantities the system suggests.

5. Improved Business Visibility

The dashboard presents sales analysis, demand forecasts, inventory alerts and restocking recommendations in one application, allowing users to access relevant business information more easily.

6. Improved Decision-Making

The chatbot and dashboard make retail information easier to access and interpret, supporting decisions based on historical data rather than relying entirely on personal judgement.

7. Improved Product Availability

By helping the business identify potential shortages and plan restocking activities, the system is expected to support better availability of everyday products for customers.

Overall, these benefits are intended outcomes of the proposed system. Achieving them would require evaluation using real business data over an appropriate operating period.

2.6 Project Scope

The scope of Retail-X-AI covers developing a Python-based inventory demand forecasting and smart restocking decision-support system for a fictional local spaza shop.

2.6.1 In-Scope Activities

The project includes the following:

1. Analysis and preparation of historical retail sales and inventory datasets.
2. Development of a machine learning model for product demand forecasting.
3. Analysis and visualisation of historical sales information.
4. Identification of potential stockout risks using available inventory and demand information.
5. Generation of advisory restocking recommendations.
6. Development of a Streamlit dashboard to display sales, forecasting, and inventory information.
7. Development of an AI chatbot using predefined intent recognition and supported data queries.
8. Development and organisation of database schemas and supporting data structures.

9. Functional testing, AI model evaluation and application integration testing.
10. Documenting the system, its implementation, and its limitations.

2.6.2 Out-of-Scope Activities

The following activities are outside the current implementation:

1. Integration with an actual spaza shop's point-of-sale system.
2. Live synchronisation with physical store inventory.
3. Automatic purchasing of products from suppliers.
4. Direct communication or transaction processing with suppliers.
5. Deployment and operation across multiple real retail businesses.
6. An external large language model-powered chatbot.
7. Product expiry monitoring using actual expiry dates, as the selected dataset does not contain expiry-date information.
8. Verification of the proposed business improvement targets through actual shop operations.

2.6.3 Scope Limitations

The system uses historical retail datasets rather than live transaction records from an operating spaza shop. Consequently, the forecasting results and restocking recommendations depend on the quality, structure and limitations of the available dataset.

The system is intended to support inventory-related decisions. It does not replace the shop owner's responsibility to review recommendations and make final purchasing decisions.

CHAPTER 3: REQUIREMENTS ANALYSIS

3.1 Introduction

Requirements analysis identifies the functions and conditions that the Retail-X-AI system must satisfy to address the inventory management challenges discussed in Chapter 2. The requirements are based on the proposed needs of a fictional spaza shop and the features available in the developed Streamlit application.

The requirements are divided into functional requirements, non-functional requirements, business rules, user stories and acceptance criteria. Functional requirements describe what the system should do, while non-functional requirements describe the system's expected quality and performance.

3.2 Functional Requirements

Functional requirements describe the main operations and services Retail-X-AI provides.

The following table identifies the requirements and their implementation status.

ID	Functional requirement	Description	Status
FR-001	Load retail data	The system shall load prepared retail sales and inventory data from CSV files for analysis.	Implemented
FR-002	Display dashboard	The system shall display an overview of relevant retail information through a dashboard.	Implemented
FR-003	Analyse sales	The system shall provide sales analysis to help users understand product sales patterns.	Implemented

FR-004	Forecast demand	The system shall generate predicted product demand using the implemented forecasting process.	Implemented
FR-005	Identify stock risks	The system shall display inventory alerts that highlight potential stock-related risks.	Implemented
FR-006	Recommend restocking	The system shall generate recommendations to support inventory restocking decisions.	Implemented
FR-007	Identify slow-moving products	The system should help identify products with low sales activity.	Requires verification
FR-008	Monitor product expiry	The system should identify products approaching their expiry dates.	Not supported by current dataset
FR-009	Provide an AI chatbot	The system shall provide a chatbot interface that responds to supported inventory and sales queries.	Implemented
FR-010	Display model performance	The system should provide information about forecasting model performance.	Present in application menu, testing not yet confirmed

The implementation status describes the current project evidence. It does not mean that every requirement has been fully validated under real shop operating conditions.

The original project plan also included product management, user authentication, sales recording and live inventory management. These were planned features, but they should not be reported as completed functions of the current Streamlit application.

3.3 Non-Functional Requirements

Non-functional requirements describe the quality characteristics expected from the Retail-X-AI application.

ID	Requirement	Description
NFR-001	Usability	The application should have a clear and understandable interface that allows users to access its main features.
NFR-002	Performance	The application should process the prepared dataset and display results within a reasonable period.
NFR-003	Reliability	The application should produce consistent results when the same input data and processing conditions are used.

NFR-004	Data accuracy	The system should use correctly formatted and relevant data to reduce errors during analysis and forecasting.
NFR-005	Maintainability	The source code should be organised into manageable files and components to make future updates easier.
NFR-006	Compatibility	The application should be accessible through a supported web browser when the Streamlit application is running.
NFR-007	Security	The system should avoid exposing sensitive information and should handle input data responsibly.
NFR-008	Availability	The application should be accessible when the application environment and required files are available.

These requirements describe the expected quality of the system. They should not automatically be interpreted as proof that every quality characteristic has passed formal testing.

3.4 Business Rules

Business rules define the conditions that should guide inventory management and the recommendations Retail-X-AI generates.

ID	Business rule	Explanation
BR-001	Valid product information	Product records should contain valid product identification information.
BR-002	Non-negative inventory	Inventory quantities should not be negative.
BR-003	Stock availability	In a live sales environment, a sale should not exceed the available inventory quantity.
BR-004	Expiry monitoring	Products approaching their expiry dates should generate an alert when expiry information is available.
BR-005	Data-based recommendations	Restocking recommendations should be based on available inventory information and the defined forecasting rules.
BR-006	Forecasting data	Demand forecasts should be generated using the available historical retail data.
BR-007	Inventory risk identification	Inventory alerts should be based on the stock and demand information available to the system.

Some of these rules describe the intended behaviour of a future live inventory system. For example, BR-003 requires a live sales and inventory transaction process, which the CSV-based application does not currently provide. BR-004 also cannot be fully applied because the selected dataset does not contain product expiry dates.

3.5 User Stories

User stories describe the system from the perspective of people who may interact with or benefit from Retail-X-AI.

ID	User story
US-001	As a shop owner, I want to view a dashboard so that I can understand the general inventory and sales situation.
US-002	As a shop owner, I want to view sales analysis so that I can identify product sales patterns.
US-003	As a shop owner, I want to view demand forecasts so that I can make better purchasing decisions.
US-004	As a shop manager, I want to view inventory alerts so that I can identify possible stock shortages.
US-005	As a shop owner, I want to view restocking recommendations so that I can make informed decisions about replenishing stock.
US-006	As a shop owner, I want to ask the chatbot questions so that I can access supported sales and inventory information more easily.
US-007	As a shop manager, I want to identify slow-moving products so that I can review products with low sales activity.
US-008	As a shop owner, I want to monitor products approaching expiry so that I can reduce potential expiry-related losses.

These user stories represent the business needs and intended use of the system. Some describe future functionality that is not available in the current implementation.

3.6 Acceptance Criteria

Acceptance criteria specify the conditions that must be met before a requirement can be considered satisfactory.

Requirement	Acceptance criteria

FR-001: Load retail data	The application successfully reads the required CSV files and makes the available data accessible for analysis.
FR-002: Dashboard	The dashboard opens and displays the available retail information without an application error.
FR-003: Sales analysis	The sales analysis page opens and displays the available sales information and analysis.
FR-004: Demand forecasting	The forecasting page opens and displays generated demand predictions from the available forecast data.
FR-005: Inventory alerts	The inventory alerts page opens and displays the available inventory risk information.
FR-006: Smart restocking	The restocking page opens and displays the generated recommendations.
FR-007: Slow-moving products	The system identifies products with low sales activity using a defined and tested method.
FR-008: Expiry monitoring	The system uses product expiry dates to identify products approaching expiry and generate alerts.
FR-009: AI chatbot	The chatbot interface opens and responds to supported questions using its available intent and data-query functionality.
FR-010: Model performance	The model performance page displays relevant evaluation information and is verified through testing.

The first six requirements and the chatbot interface were manually confirmed to open and work during application checks. The Model Performance page still requires verification. The slow-moving product and expiry monitoring requirements also require separate verification before they can be reported as fully working features.

3.7 Requirements Summary

The requirements analysis establishes the main functions and quality expectations for Retail-X-AI. The system focuses on analysing prepared retail data, displaying sales information, forecasting demand, identifying inventory risks, providing restocking recommendations and allowing users to interact with a chatbot. The requirements also identify limitations between the original project proposal and the current application. Features such as live sales recording, user authentication, direct inventory updates and expiry monitoring are not confirmed as completed functions. These limitations are considered when evaluating the system in the later testing and results chapter.

CHAPTER 4: SYSTEM ANALYSIS AND DESIGN

4.1 System Overview

Retail-X-AI is designed to help small retail businesses understand their sales and inventory information. The system uses historical retail data and Python-based analytical processes to generate demand forecasts, identify possible stock risks and provide restocking recommendations.

The application provides a Streamlit-based user interface that lets users access different sections of the system. These include the dashboard, sales analysis, AI forecasting, smart restocking, inventory alerts and an AI chatbot. The application menu also includes a Model Performance page, although its operation still requires verification.

The system is designed around the principle of using available retail information to support business decisions. It does not automatically purchase products or replace the shop owner's responsibility for making purchasing decisions.

4.2 System Actors

System actors represent the external users or entities that interact with Retail-X-AI or provide information to it.

The main actors identified during system analysis are:

Actor	**Description**	**Interaction with the system**
Shop Owner	The person responsible for the overall operation of the spaza shop.	Reviews sales information, forecasts, alerts and restocking recommendations.
Shop Manager	Assists with daily shop operations and inventory decisions.	Uses available information to monitor stock and review recommendations.
Shop Employee	Assists with sales and inventory-related activities.	May provide sales and inventory information in a future data capture implementation.

Historical Retail Dataset	External source of historical retail information.	Supplies data used for sales analysis, inventory analysis and demand forecasting.
System Administrator	Responsible for maintaining the application environment.	Supports application configuration and technical maintenance.

The shop owner or manager is the primary system actor. The historical retail dataset is an external data source, not a person using the application.

4.3 Use Case Analysis

Use case analysis describes the main activities that users are expected to perform when interacting with Retail-X-AI.

The repository identifies six main use cases:

Use Case ID	**Use Case**	**Description**
UC-01	Capture Sales and Stock Data	Allows sales and inventory information to be supplied to the system.
UC-02	Generate Demand Forecast	Uses historical retail information to estimate future product demand.
UC-03	Calculate Recommended Reorder Quantities	Calculates suggested quantities for replenishing stock.
UC-04	Identify Slow-Moving and Near-Expiry Products	Identifies products requiring attention based on sales movement and expiry information where available.
UC-05	Interact with Natural-Language AI Assistant	Allows users to submit questions and receive supported responses.
UC-06	View Interactive Dashboard	Presents relevant sales, inventory and analytical information.

These use cases describe the intended system functionality. However, the current application does not provide all the proposed operations. For example, the current CSV-based application analyses prepared data rather than providing a complete live sales capture and inventory update process.

The detailed use case specifications are available in the repository under:

documentation/02-requirements/use-cases/M04-04-use-case-specifications.md

4.4 System Context Diagram

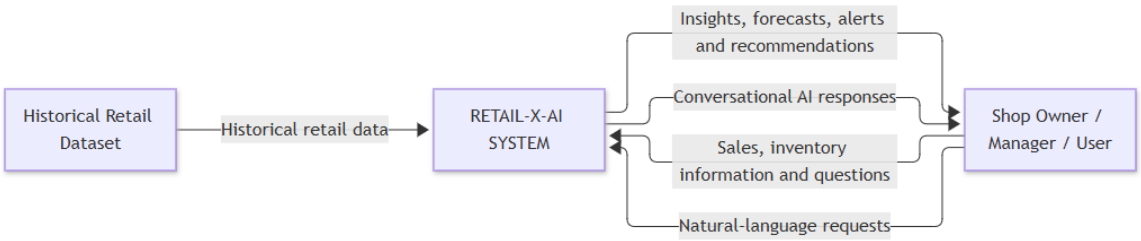
The system context diagram provides a high-level view of Retail-X-AI and its relationship with external entities. It establishes the system boundary and identifies the main information entering and leaving the application.

The diagram identifies two external entities:

1. **Shop Owner / Manager / User:** Provides information and questions to the system and receives forecasts, alerts, recommendations and analytical results.
2. **Historical Retail Dataset:** Supplies historical retail information used by the system for analysis and forecasting.

The main inputs to Retail-X-AI include historical retail data, sales and inventory information, and user questions. The system produces sales insights, demand forecasts, stockout-risk alerts, restocking recommendations, and chatbot responses.

4.2: Retail-X-AI System Context Diagram



---

diagrams/architecture/m05-01-system-context-diagram.md

The context diagram establishes the overall system boundary without showing the internal processing steps. Data flow diagrams explain these.

4.5 Data Flow Diagrams

Data Flow Diagrams (DFDs) illustrate how information moves through a system, including its external entities, internal processes and logical data stores.

4.5.1 DFD Level 0

The Level 0 DFD provides an overview of the major processes within Retail-X-AI. It expands the system context diagram by identifying seven main processes.

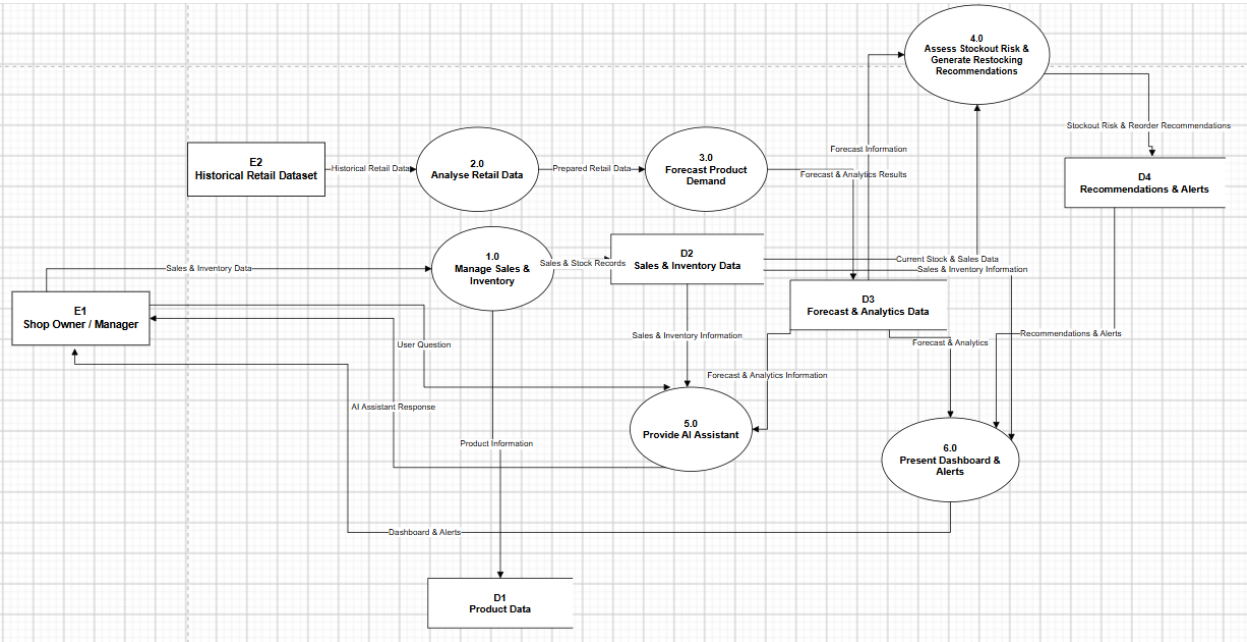
Process ID	Process	Description
1.0	Manage Sales and Inventory Data	Handles the supply, validation and organization of sales and inventory information.
2.0	Analyze Retail Data	Analyses available product, sales and inventory information.
3.0	Forecast Product Demand	Generates estimated future demand using historical retail information.
4.0	Assess Stockout Risk	Identifies possible stock shortages using available stock and demand information.
5.0	Generate Smart Restocking Recommendations	Produces suggested reorder quantities to support purchasing decisions.
6.0	Provide AI Assistant	Processes user questions and provides responses using available retail information.
7.0	Present Dashboard and Alerts	Displays analytical results, forecasts, alerts and recommendations.

The DFD also identifies four logical data stores:

- D1: Product Data
- D2: Sales and Inventory Data
- D3: Forecast and Analytics Data
- D4: Recommendations and Alerts

These data stores represent the logical information the proposed system requires. They should not be interpreted as proof that all four are currently implemented as connected database tables.

4.3: Retail-X-AI DFD Level 0



diagrams/dfd/M05-02-DFD-Level-0.png

4.5.2 DFD Level 1

The Level 1 DFD provides a more detailed breakdown of the processes identified in the Level 0 diagram.

The main process groups include:

Sales and Inventory Management

This process group covers recording sales information, updating inventory levels, managing product information and checking current stock.

Retail Data Preparation and Analysis

This process group covers importing historical retail data, preparing records, analysing sales trends and examining inventory trends.

Demand Forecasting

The forecasting process prepares the required information, generates demand predictions and makes the resulting forecasts available for further analysis and reporting.

The initial forecasting target identified in the project documentation is Units Sold. The data analysis also identifies Demand Forecast, Units Ordered and Inventory Level as potential data leakage risks. These fields must be assessed carefully before being used as model inputs.

Stockout Risk and Smart Restocking

This process group assesses potential stock shortages, identifies restocking needs, calculates suggested reorder quantities and makes recommendations available to users.

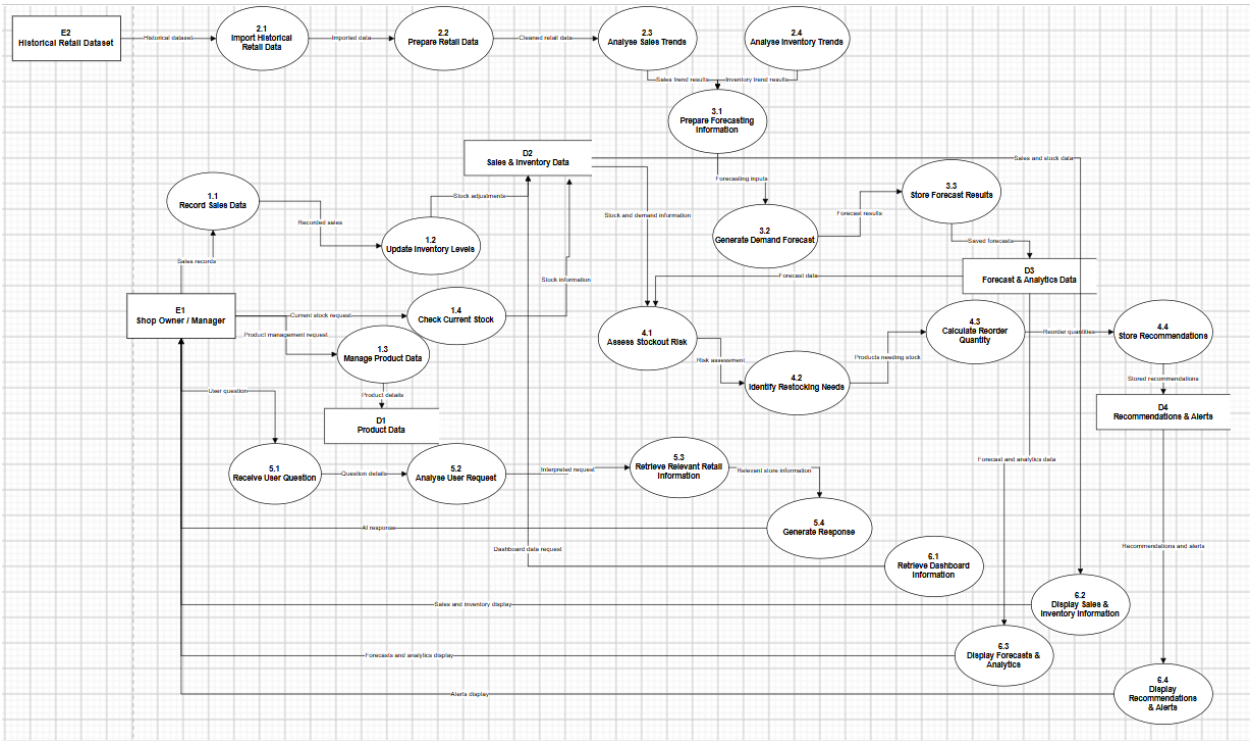
AI Assistant

The assistant receives user questions, analyses the request, retrieves relevant information where available and generates a response.

Dashboard and Reporting

This process group retrieves and displays available sales information, inventory information, forecasts, alerts and restocking recommendations.

4: Retail-X-AI DFD Level 1



diagrams/dfd/M05-03-DFD-Level-1.png

The Level 1 diagram provides a more detailed explanation of the information processing activities within the proposed system.

4.6 System Architecture

The Retail-X-AI repository contains a logical system architecture divided into five main layers.

4.6.1 Presentation Layer

This layer represents the user-facing interface. It includes the dashboard, alerts interface and conversational assistant interface through which users access the system's functionality.

4.6.2 Application Layer

This layer coordinates user requests and application operations. The logical design includes request validation, authentication and access control.

4.6.3 Domain Services Layer

This layer contains the main business processes of Retail-X-AI, including:

- Sales and inventory management
- Historical data import and validation
- Retail data analysis
- Demand forecasting
- Stockout-risk assessment
- Restocking recommendations
- Dashboard reporting
- AI assistant coordination

4.6.4 AI and Machine Learning Layer

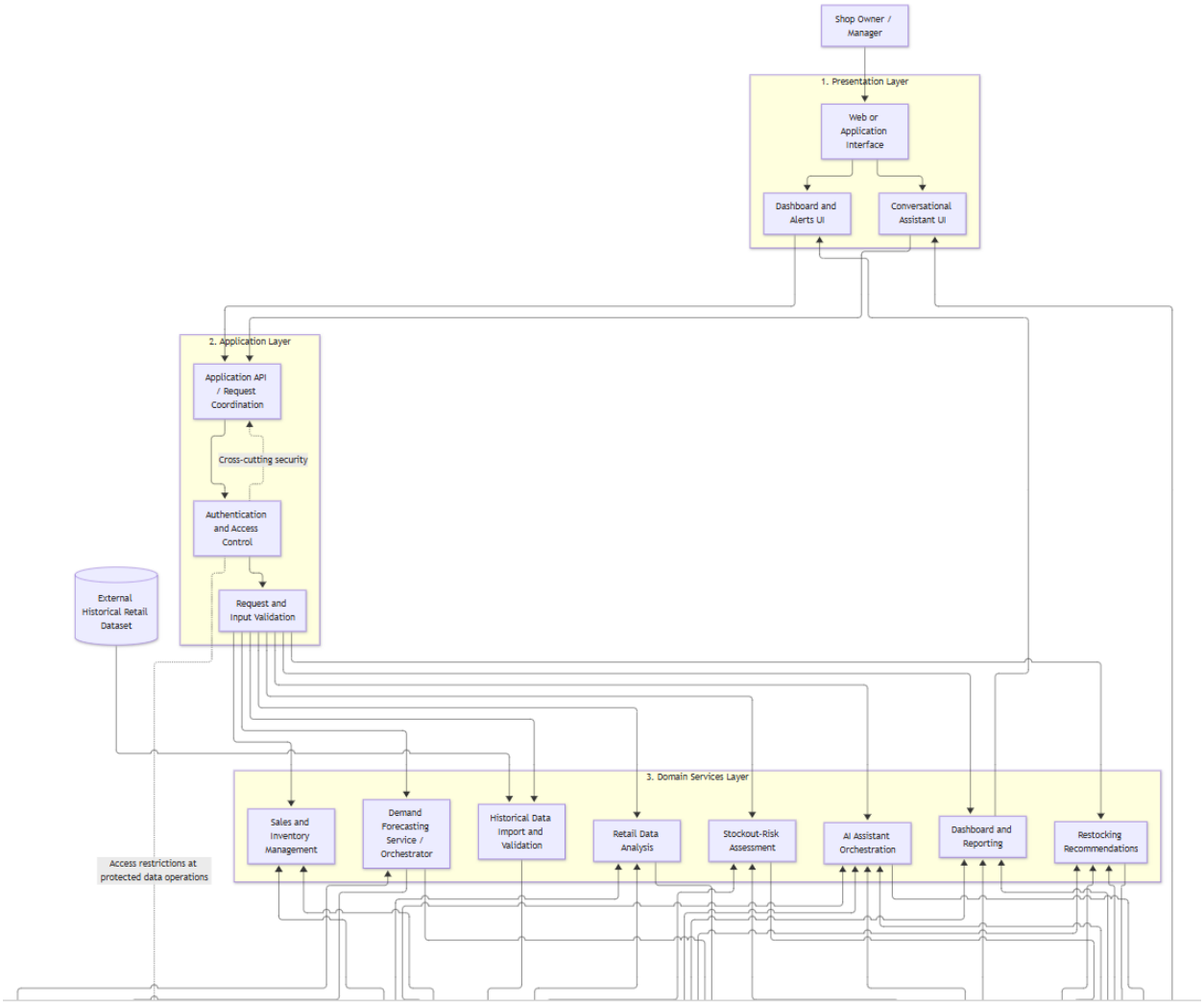
This layer represents the data preparation and machine learning activities required for forecasting. It includes feature preparation, the forecasting model, and the assistant response-processing component.

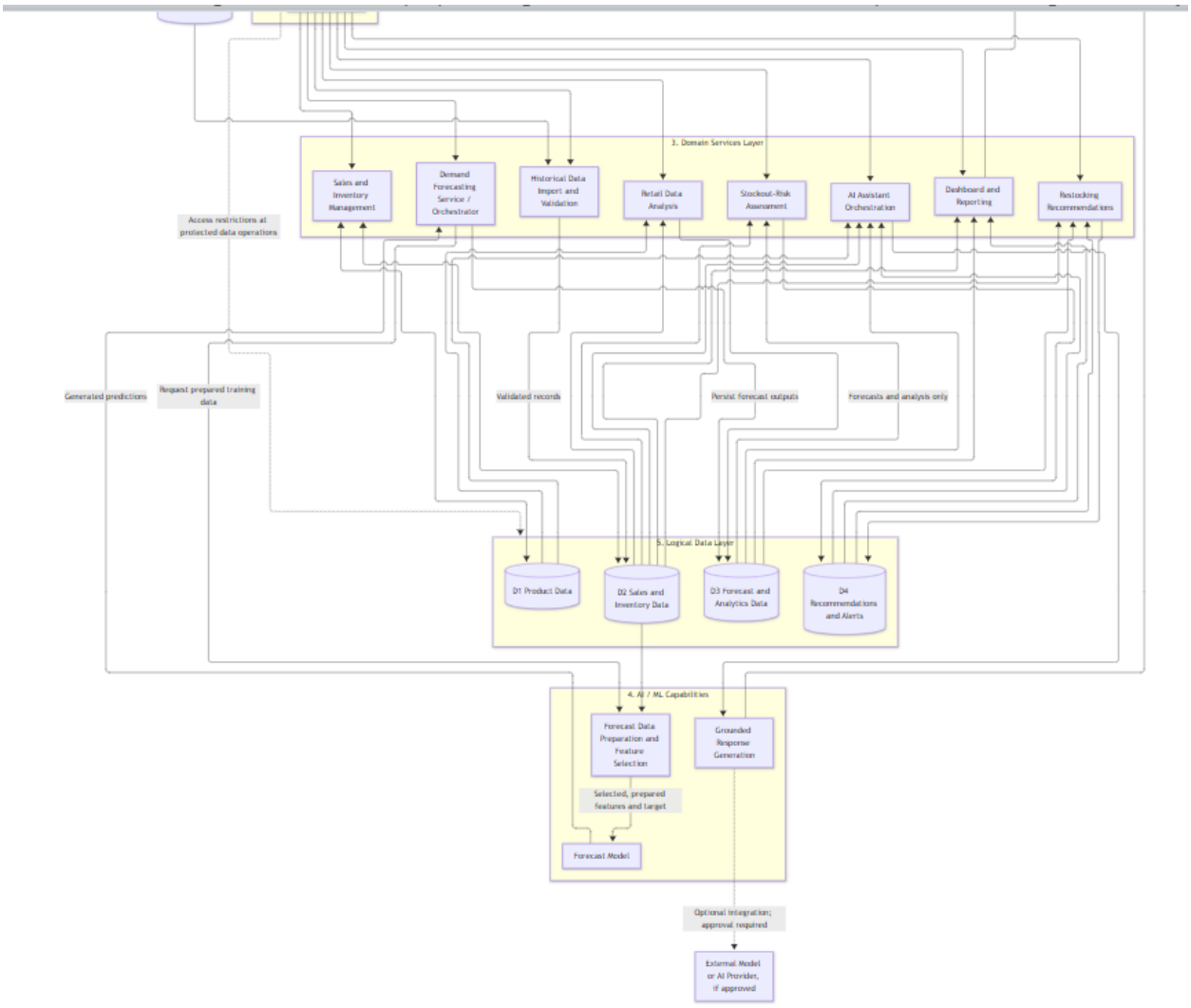
4.6.5 Logical Data Layer

The logical data layer represents the information the system requires. It includes product data, sales and inventory data, forecast and analytics results, and recommendations and alerts.

The architecture also identifies an external historical retail dataset as an input source.

Figure 4.5: Retail-X-AI Logical System Architecture





diagrams/architecture/M06-01-system-architecture.md

The architecture is a logical design rather than a confirmed physical deployment diagram. It describes how the system components are intended to interact. The current implementation uses a Streamlit application and prepared CSV files. The logical architecture should therefore not be interpreted as proof that every proposed application service, authentication component or database connection is operational.

4.7 User Interface Design

The user interface design provides a clear way for shop owners and managers to access retail information and analytical features.

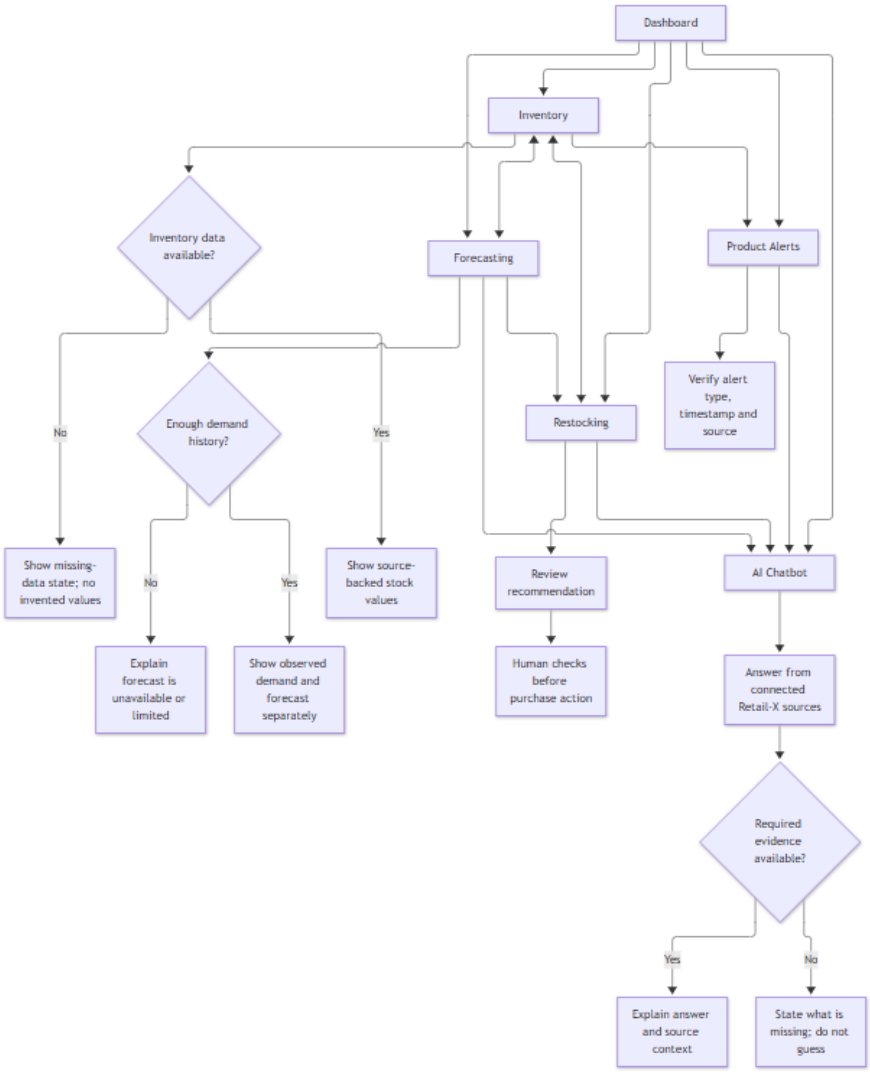
The repository contains static wireframes that describe the proposed interface. The wireframes use a dark-mode design with a consistent navigation area and separate screens for the main system functions.

The proposed interface includes six main screens:

Screen	Purpose
Dashboard	Provides an overview of retail information and links to other system sections.
Inventory	Allows users to review product and stock information.
Forecasting	Presents historical demand information and predicted demand.
Restocking	Allows users to review suggested replenishment actions.
Product Alerts	Presents inventory-related warnings and exceptions.
AI Chatbot	Allows users to submit questions and receive supported responses.

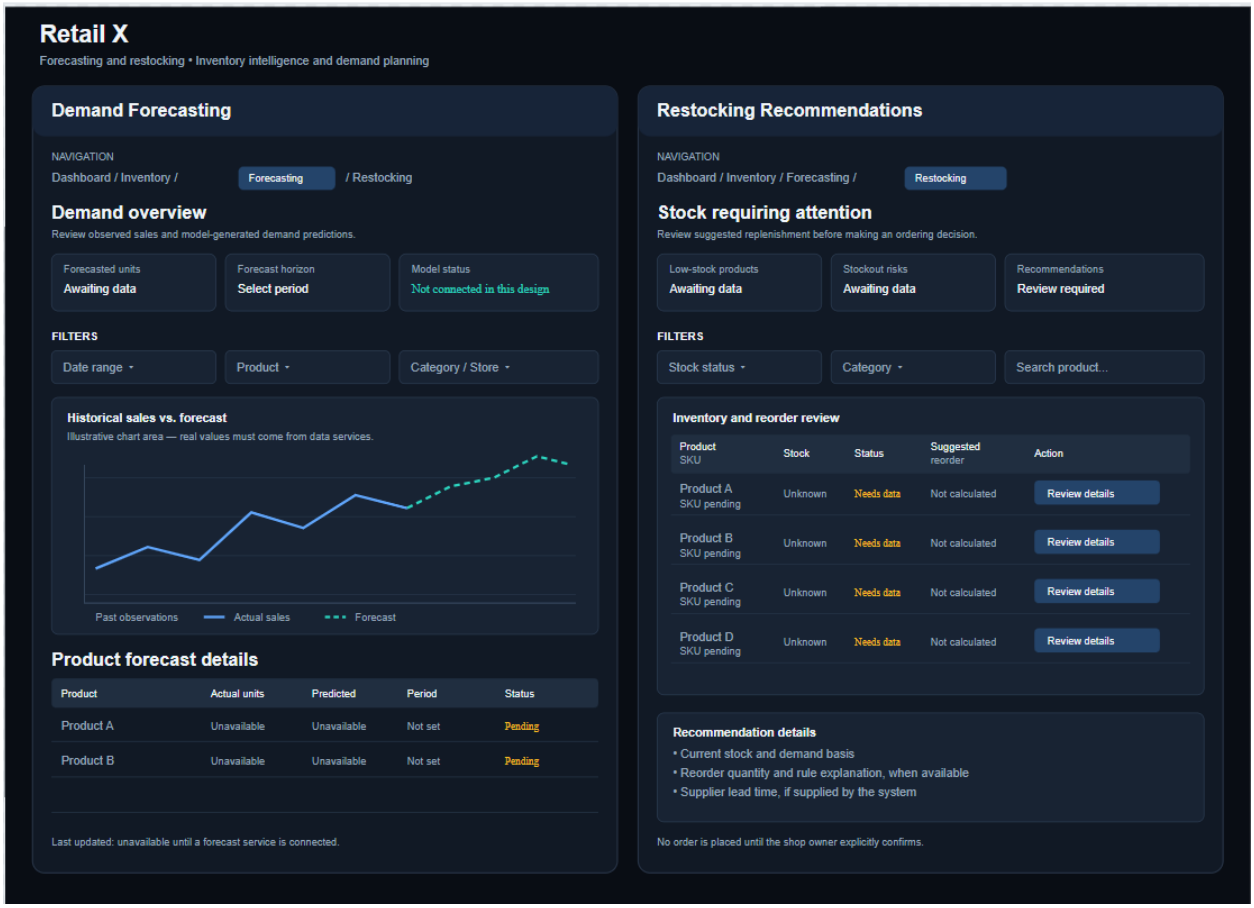
The navigation design lets users move between main sections without always returning to the dashboard.

Figure 4.6: Retail-X-AI Dashboard and Inventory Wireframes



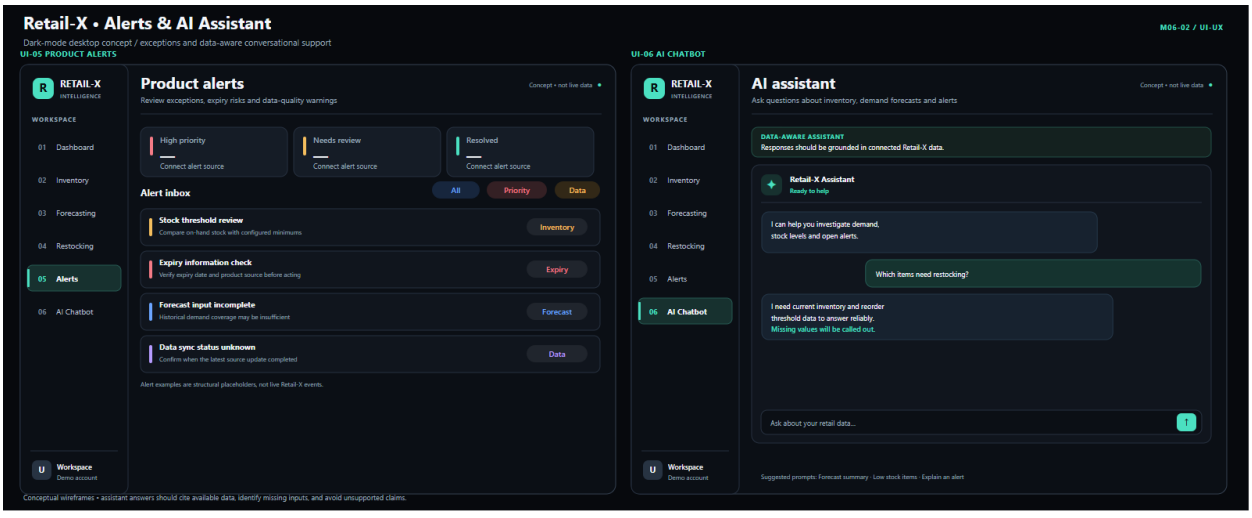
diagrams/ui-ux/M06-02-wireframes.svg

Figure 4.7: Forecasting and Restocking Wireframes



diagrams/ui-ux/M06-04-forecasting-restocking-wireframes.svg

Figure 4.8: Alerts and AI Chatbot Wireframes



diagrams/ui-ux/M06-02-wireframes-alerts-assistant.svg

These images are design wireframes and not screenshots of the running application. The actual Streamlit interface was developed separately and contains the confirmed application pages described in Chapter 7.

4.8 Database Design Overview

The database design identifies the main data entities required to organise retail information.

The repository contains an Entity Relationship Diagram (ERD) that identifies the following entities:

Entity	Description
CATEGORY	Stores product category information.
REGION	Stores regional information associated with stores.
STORE	Represents individual retail stores.
PRODUCT	Contains product details, pricing and category information.
DATE_DIM	Stores date-related information such as day, month, year and season.
INVENTORY_RECORD	Stores dated product inventory and sales information.

The ERD establishes relationships between these entities. A region can contain multiple stores, a category can classify multiple products, and stores and products can have multiple inventory records over time. Each inventory record is also associated with a date.

Figure 4.9: Retail-X-AI Entity Relationship Diagram



documentation/06-database/erd_diagram.png

The ERD represents the proposed logical data model based on the selected retail dataset. The current application uses prepared CSV files for its main analysis and dashboard functions. Therefore, the existence of the ERD and database schema does not by itself establish that the Streamlit application is operating through a live relational database.

4.9 Chapter Summary

This chapter described the system analysis and design of Retail-X-AI. It identified the main actors, use cases, system boundary, data flow processes, logical architecture, user interface design and database entities.

The diagrams provide a structured view of how Retail-X-AI is intended to process retail information and support inventory decisions. The distinction between the proposed design and the current application implementation has been maintained to ensure that the documentation accurately represents the project.

CHAPTER 5: DATASET AND DATA PREPARATION

5.1 Introduction

Data preparation is an important stage in developing Retail-X-AI because the forecasting and inventory analysis processes depend on the quality of the available data. Before developing the machine learning model, the dataset must be examined, cleaned, validated, and transformed into a suitable format.

This chapter describes the dataset used in the project, its source, structure, quality checks, cleaning activities and feature engineering process. It also explains how the prepared data was divided into training, validation and testing datasets for forecasting.

5.2 Dataset Source

Retail-X-AI uses the **Retail Store Inventory and Demand Forecasting** dataset obtained from Kaggle.

Wavelet published the dataset under the Kaggle username atomicd. The project documentation records that the dataset was downloaded on 22 September 2026 and is licensed under Apache 2.0.

Dataset source: https://www.kaggle.com/datasets/atomicd/retail-store-inventory-and-demand-forecasting

The dataset contains synthetic daily retail sales and inventory information. It represents five stores, twenty products and several product categories across four regions.

The project team modified the original Kaggle dataset before using it in Retail-X-AI. The modifications included renaming the Demand column to Demand Forecast, renaming Promotion to Holiday/Promotion and removing the Epidemic column.

The dataset is suitable for demonstrating demand forecasting and inventory analysis. However, it does not represent actual transactions from a South African spaza shop.

Therefore, the results should be understood as results obtained from the selected sample dataset rather than proven results from a real retail business.

5.3 Dataset Structure

The raw dataset contains 73,100 records and 15 columns. Each record represents daily retail information associated with a store and product.

The following table describes the dataset fields.

Column	Description
Date	Date associated with the retail record.
Store ID	Unique identifier for the store.
Product ID	Unique identifier for the product.
Category	Product category.
Region	Geographical region associated with the store.
Inventory Level	Available stock quantity.
Units Sold	Number of units sold.
Units Ordered	Number of units ordered or replenished.
Demand Forecast	Existing demand forecast value in the dataset.
Price	Selling price of the product.
Discount	Discount percentage applied.
Weather Condition	Weather condition associated with the record.
Holiday/Promotion	Indicates whether a holiday or promotion applies.
Competitor Pricing	Competitor's product price.
Seasonality	Seasonal classification.

The dataset includes five stores, twenty products, five product categories and four regions.

We inspected the actual CSV file to confirm its structure. It contains records dated from 1 January 2022 to 1 January 2024, with 731 unique dates.

5.4 Dataset Quality Assessment

A dataset quality assessment was conducted to identify issues that could affect the accuracy and reliability of the forecasting process.

The cleaning and validation report recorded the following results:

Data quality check	**Result**
Original records	73,100
Original columns	15
Missing values	0
Exact duplicate records	0
Invalid dates	0
Records with missing required values	0
Invalid numeric conversions	0
Negative inventory records	0
Negative units sold records	0
Negative demand forecast values	673

One issue identified was that 673 records contained negative values in the Demand Forecast column. Since demand cannot normally be negative, we retained these values for review rather than deleting them silently.

The forecasting feature preparation process excludes the Demand Forecast column from the model inputs. This prevents the model from depending on an existing forecast value that may not be appropriate for predicting actual sales.

The dataset verification also identified that the data is synthetic. This limits how confidently the forecasting results can be applied to real retail businesses.

5.5 Data Cleaning and Preprocessing

Data cleaning was performed using Python. The project includes a cleaning script named clean_data.py, which is located in:

python-ai/data/clean_data.py

The cleaning process ensured the dataset had a consistent structure and appropriate data types before feature preparation.

The following activities were documented:

1. Data type conversion

We converted date fields to appropriate date formats and numerical fields to suitable numeric types.

2. Missing value assessment

We checked the dataset for missing values. The cleaning report recorded no missing values in the fifteen columns.

3. Duplicate detection

We checked for exact duplicate records and found none. We removed no records due to duplication.

4. Required field validation

We identified records missing essential information, including the date, store identifier, product identifier, or units sold, for removal. However, we found none.

5. Negative value assessment

We checked for negative inventory and sales values. We identified no negative inventory or units sold records. We retained the 673 negative demand forecast values for review.

6. Schema validation

We checked the dataset to confirm that the expected fifteen columns were present and that no unexpected columns had been introduced.

The original raw dataset was preserved so that the cleaning process could be repeated without modifying the source file.

The cleaning process produced a cleaned dataset named:

dataset/processed/retail_store_inventory_clean.csv

5.6 Feature Engineering

Feature engineering transforms existing data into useful inputs that help a machine learning model identify patterns and make predictions.

For Retail-X-AI, the forecasting target was selected as Units Sold. This means that the model aims to predict the number of units expected to be sold based on the available input features.

The project used calendar features, categorical encoding and historical demand features.

5.6.1 Calendar Features

The date column was used to generate additional features that describe the time associated with each retail record.

The generated features include:

- Day of the week
- Day of the month
- Month
- Quarter
- Day of the year
- Week of the year
- Weekend indicator

These features allow the model to identify possible patterns associated with different days, months and seasons.

5.6.2 Categorical Encoding

Several dataset fields contain categories rather than numerical values. These include store identifiers, product identifiers, product categories, regions, weather conditions and seasonality.

One-hot encoding was applied to transform categorical information into numerical columns the forecasting model could use.

5.6.3 Historical Demand Features

The forecasting development process also introduced historical demand features for individual store-product combinations.

These features include:

Feature	Description
demand_lag_1	Previous observed demand.
demand_lag_7	Demand recorded seven observations earlier.
demand_lag_14	Demand recorded fourteen observations earlier.
demand_rolling_mean_7	Average demand over the previous seven observations.
demand_rolling_mean_28	Average demand over the previous twenty-eight observations.

The lag and rolling mean calculations were shifted so that the current row's actual sales value was not used to construct its own historical features.

This matters because using information from the target being predicted creates data leakage and can produce misleading evaluation results.

5.6.4 Excluded Features

Three columns were excluded from the forecasting model inputs:

1. Demand Forecast
2. Units Ordered
3. Inventory Level

We excluded these fields because of potential target leakage or uncertainty about whether their values would be available when generating a forecast.

The feature preparation documentation initially recorded 53 engineered features. A later forecasting development stage introduced five historical demand features, resulting in 58 features for the expanded forecasting pipeline.

5.7 Training, Validation and Testing Data

After cleaning and preparing the data, we divided the dataset into three groups for machine learning development and evaluation.

We used a chronological split instead of randomly distributing the records.

This approach ensures that older records are used for training while later records are reserved for validation and testing. It better represents a forecasting situation in which a model uses historical information to predict future demand.

The reproducible pipeline report recorded the following split:

Dataset	**Number of records**	**Purpose**
Training	51,100	Used to train the forecasting model.
Validation	11,000	Used to assess and compare model performance during development.
Testing	11,000	Used for final evaluation on unseen records.
Total	73,100	**Complete dataset**

The splitting process was validated using six checks:

- Chronological ordering was maintained.
- All three datasets contained identical feature columns.
- No dates were shared between the training, validation and testing datasets.
- The split record counts matched the combined dataset.
- All usable records were retained.
- No missing target values were found.

The reproducible pipeline report recorded all six checks as passed.

The data preparation process can be repeated using the project's Python environment and the following command:

python python-ai/data/prepare_temporal_splits.py

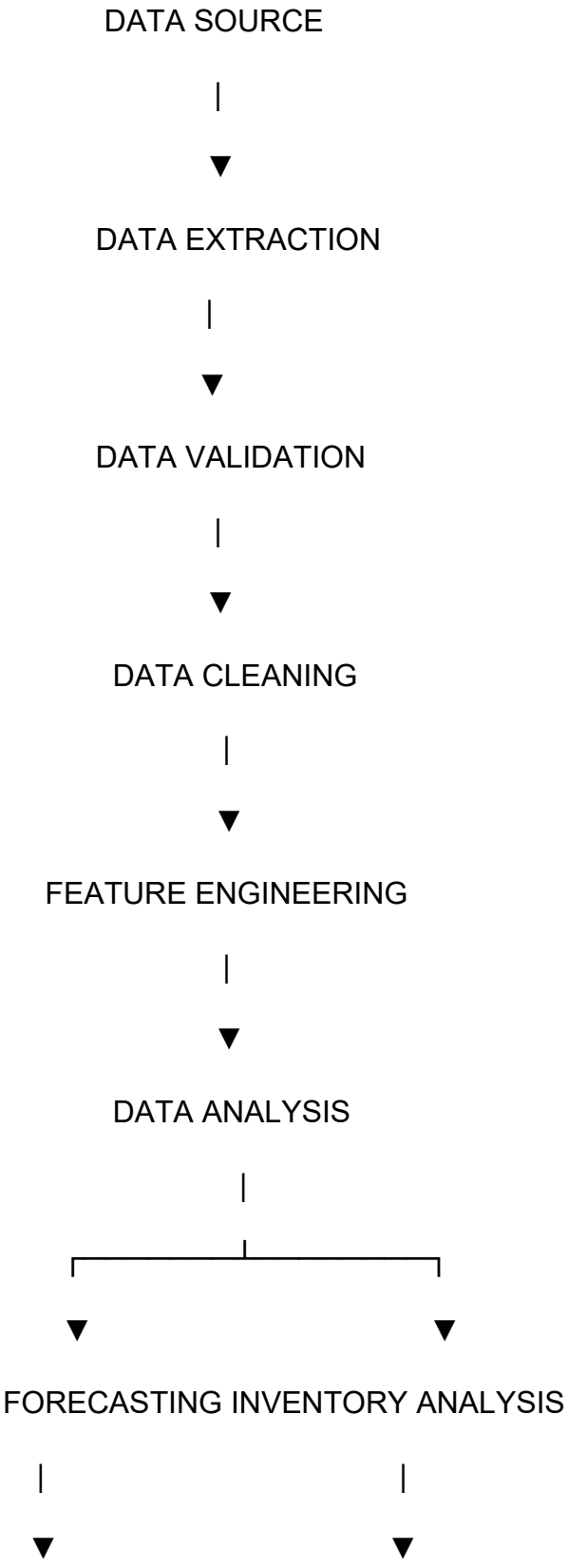
This command cleans the data, prepares features, performs chronological splitting, and validates the output.

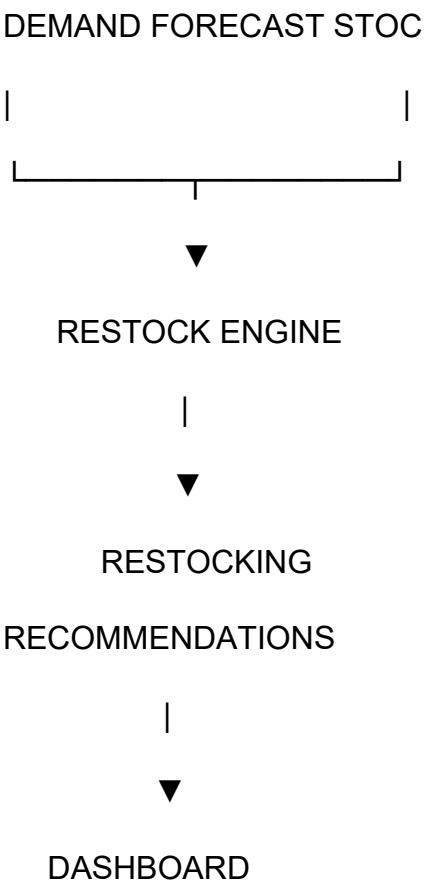
5.8 Data Preparation Pipeline

The Retail-X-AI data preparation process can be summarised as follows:

1. Load the raw retail dataset.
2. Inspect the dataset structure and data types.
3. Validate the required columns.
4. Check for missing values, duplicates and invalid values.
5. Convert dates and numerical fields into appropriate formats.
6. Prepare the cleaned dataset.
7. Generate calendar and categorical features.
8. Generate historical demand features.
9. Exclude unsuitable forecasting inputs.
10. Divide the dataset chronologically into training, validation and testing groups.
11. Validate the generated datasets before model development.

Figure 5.1: Retail-X-AI Data Preparation Pipeline





5.9 Dataset Limitations

Although the selected dataset provides sufficient records for demonstrating forecasting and inventory analysis, several limitations must be considered.

Synthetic information: The dataset was synthetically generated and may not represent customers' actual purchasing behaviour at South African spaza shops.

Limited business information: The dataset does not contain actual supplier information, purchase costs or complete business transaction details.

No expiry dates: The dataset has no product expiry date column. Therefore, the dataset cannot directly support reliable product expiry monitoring.

Limited store representation: The dataset represents five stores and twenty products. It does not cover the full range of products and operational conditions found in real retail businesses.

Demand forecast quality: The dataset contains 673 negative values in the existing demand forecast column. These values require attention, and that column is excluded from the forecasting model inputs.

Real-world generalisation: A model trained using this dataset may not perform in the same way when applied to real spaza shop data. Additional data from actual retail operations would be required to evaluate its usefulness in a real business environment.

These limitations are important when interpreting the forecasting results and recommendations presented in later chapters.

5.10 Summary

This chapter described the dataset used in Retail-X-AI and the preparation activities performed before machine learning development. The selected Kaggle dataset contains 73,100 records and fifteen columns covering sales, inventory, products, stores and other retail-related information.

Python was used to clean and validate the dataset, generate forecasting features, and split the data into chronological training, validation, and testing groups. The resulting pipeline provides a reproducible foundation for developing and evaluating the demand forecasting model.

However, the synthetic nature of the dataset and the absence of actual spaza shop transactions, supplier information and expiry dates limit the conclusions that can be drawn from the results.

AI DEVELOPMENT AND IMPLEMENTATION

6.1 Introduction

This chapter explains the development and implementation of the artificial intelligence components used in the Retail-X-AI system. These components analyse historical retail data, predict product demand, identify potential stock shortages, and recommend restocking.

The system uses Python and machine learning techniques to process historical sales and inventory information. Ridge Regression serves as the main demand-forecasting model. The inventory analysis and smart restocking components then use the forecasting results to help the shop make better inventory decisions.

The chapter also discusses the technologies used, model training and evaluation, time series analysis, inventory calculations, chatbot functionality and the limitations of the implemented solution.

6.2 Development Environment and Technologies

The Retail-X-AI system was developed using Python and several supporting libraries. These technologies were selected because they support data processing, machine learning, analysis and the development of interactive applications.

The main technologies include:

Technology	**Purpose**
Python	Main programming language used for AI development and data processing.
Pandas	Used to load, organise and manipulate dataset records.
NumPy	Used for numerical calculations and data operations.
Scikit-learn	Used to train, compare and evaluate machine learning models.
Joblib	Used to save and load the trained forecasting model.

Matplotlib	Used for data visualization and analysis.
Streamlit	Used to develop the interactive system dashboard.
SQLite	Used by the database service to store and manage inventory records.

These technologies support different parts of the system. Python and its machine learning libraries handle the forecasting and data analysis, while Streamlit provides the interface through which users can view information and interact with the system.

6.3 Demand Forecasting Using Ridge Regression

Demand forecasting is one of Retail-X-AI's main AI functions. It involves analysing historical sales patterns to estimate expected product demand.

The project uses Ridge Regression as its main forecasting model. Ridge Regression is a supervised machine learning algorithm that predicts numerical values by learning relationships between input features and a target variable.

For this project, the target variable is **Units Sold**. The model uses historical sales information and other prepared features to estimate the number of units that may be sold.

6.3.1 Feature Preparation

Before training the model, the dataset was prepared to ensure the machine learning algorithm could use the information.

The features include:

1. Calendar information such as day, month and year.
2. Product and store information.
3. Holiday and promotion information.
4. Historical demand information.
5. Previous sales values.
6. Rolling averages calculated from historical sales.

The project also uses historical features such as:

Feature	Description
Lag 1	Previous sales observation.
Lag 7	Sales observation from seven periods earlier.
Lag 14	Sales observation from fourteen periods earlier.
Rolling Mean 7	Average sales over the previous seven periods.
Rolling Mean 28	Average sales over the previous twenty-eight periods.

These features help the model recognise patterns in historical sales instead of relying only on the current record.

The final forecasting dataset contains 58 input features.

6.3.2 Model Selection

We evaluated different machine learning approaches to identify a suitable forecasting model.

The models compared were:

- Ridge Regression
- Random Forest
- HistGradientBoosting
- Historical Average Baseline

We selected Ridge Regression as the main model because it produced the lowest validation Mean Absolute Error among the evaluated models.

6.3.3 Model Training

The dataset was divided into training, validation and testing sections using a chronological approach.

Dataset	Number of records
Training	51,100
Validation	11,000
Testing	11,000
Total	73,100

The chronological split allows the model to learn from earlier records before being evaluated on later records.

The Ridge Regression model was trained using StandardScaler for feature scaling and an alpha value of 10.0 for regularisation.

The trained model was saved using Joblib so it could be loaded again without repeating the training process.

Saved model: python-ai/forecasting/artifacts/retail_demand_ridge_model.joblib

6.3.4 Model Comparison Results

The following table presents the validation results obtained during model comparison.

Model	MAE	RMSE
Ridge Regression	89.7752	109.7737
Historical Average	89.7934	109.8169
HistGradientBoosting	89.8343	109.8009
Random Forest	90.1691	110.0959

Ridge Regression achieved the lowest MAE and RMSE among the evaluated models. However, its results were close to those of the historical average baseline. This means that although Ridge Regression performed best in this comparison, the improvement over the baseline was relatively small.

6.3.5 Final Model Evaluation

After model selection and training, the Ridge Regression model was evaluated using the separate testing dataset.

The final test results were:

Evaluation measure	Result
Mean Absolute Error (MAE)	88.453325
Root Mean Squared Error (RMSE)	107.996727
Test records	11,000
Unique test dates	110
Input features	58

MAE measures the average absolute difference between actual sales values and model predictions. RMSE measures prediction errors and weights larger errors more heavily.

These results indicate the model's performance on the testing dataset. However, the error values should be considered when interpreting its predictions. The model should not be assumed to predict every product's demand accurately.

The test output is stored in:

python-ai/forecasting/outputs/retail_demand_forecasts_m11_06.csv

The testing predictions use historical information from the dataset in a rolling one-step-ahead evaluation. They should not be described as forecasts for dates beyond the available dataset.

6.4 Time Series Analysis

Time series analysis involves examining data collected over time to identify patterns and changes. It is relevant to Retail-X-AI because product sales and demand change according to different periods.

For example, a product may experience higher sales during certain months, weekends, holidays or promotional periods. Analysing these changes can help a retailer understand how demand behaves over time.

The project uses date-related information and historical sales features to support demand forecasting.

The time-related features include:

1. Day information.
2. Month information.
3. Year information.
4. Previous sales observations.
5. Rolling averages over different historical periods.

These features give the forecasting model information about past sales behaviour.

The chronological training, validation and testing process also supports time-based evaluation. Earlier records are used for training, while later records are reserved for validation and testing.

This approach matters because randomly mixing historical records could let information from later periods influence training.

6.5 Inventory Analysis and Stockout Risk Detection

The inventory analysis component uses forecast demand and current inventory information to identify products that may experience stock shortages.

The purpose is to help the shop identify when its available stock may not be sufficient to meet expected customer demand.

The system considers:

- Forecast daily demand.

- Current inventory level.
- Supplier lead time.
- Safety stock requirements.
- Review period.

The project uses the following assumptions:

Parameter	Value
Supplier lead time	3 days
Review period	1 day
Safety stock period	1 day

These values are system assumptions and do not represent verified delivery arrangements from an actual supplier.

6.5.1 Safety Stock Calculation

Safety stock is additional inventory kept to reduce the possibility of running out of stock.

The project calculates safety stock using:

Safety Stock = Daily Forecast Demand × Safety Stock Days

6.5.2 Reorder Point Calculation

The reorder point indicates the inventory level at which the shop should consider ordering more stock.

The formula is:

Reorder Point = Daily Forecast Demand × Lead Time + Safety Stock

6.5.3 Target Stock Calculation

Target stock represents the inventory level the system aims to maintain based on the lead time, review period and safety stock.

Target Stock = Daily Forecast Demand × (Lead Time + Review Period) + Safety Stock

6.5.4 Stock Risk Classification

The system uses stock risk categories to communicate inventory conditions.

Status	Meaning
STOCKOUT_RISK	Inventory may not be sufficient to meet expected demand.
WATCH	Inventory requires attention and monitoring.
NO_STOCKOUT_RISK	Current inventory does not indicate an immediate stockout risk.

These classifications help the shop identify products that may require attention.

6.6 Smart Restocking Recommendations

The smart restocking component uses the inventory analysis results to calculate how much additional stock may be required.

The system calculates the suggested order quantity using:

Suggested Order Quantity = Target Stock − Current Inventory

The recommendation is based on the difference between the target inventory level and the available stock.

The system uses the following recommendation statuses:

Status	Meaning
REORDER_NOW	The available inventory indicates that more stock should be ordered.
WATCH	The product should be monitored because its inventory position may require attention.
HEALTHY	The current inventory level does not indicate an immediate need to reorder.
DATA_REQUIRED	Additional information is required before a recommendation can be calculated.
INSUFFICIENT_HISTORY	There is not enough historical information to support a reliable recommendation.

The recommendations are advisory. The system does not automatically place orders, communicate with suppliers or purchase inventory.

This feature helps the shop make more informed restocking decisions by considering forecast demand instead of relying only on manual judgement.

6.7 Natural Language Processing and AI Chatbot

Retail-X-AI includes a chatbot that allows users to interact with the system through text-based questions.

The chatbot uses predefined intent recognition and pattern matching to identify the type of question the user enters. It then uses supported data queries to retrieve relevant information and respond.

For example, a user may ask about inventory levels, sales information or products that require attention.

The chatbot supports interaction with available system information without requiring the user to navigate through every dashboard page.

The chatbot is not based on a large language model. It uses predefined intents and responses together with data queries. Its responses are therefore limited to the questions and information supported by its implementation.

The chatbot provides an additional way for users to access information and understand the system's inventory results.

6.8 Techniques Used to Support Forecasting Accuracy

Several techniques were used to prepare the dataset and support the forecasting process.

Data cleaning: The dataset was checked and prepared before training the model. This helps reduce problems caused by invalid or inconsistent records.

Feature engineering: We created additional features from existing information, including calendar features and historical sales values. These features provide the model with more information about demand patterns.

Feature scaling: StandardScaler was used as part of the Ridge Regression pipeline to scale numerical input features.

Chronological data splitting: The dataset was split chronologically rather than randomly. This helps maintain the order of historical records during model evaluation.

Model comparison: Different machine learning approaches were compared using MAE and RMSE to identify the model with the best validation performance.

These techniques support the forecasting process, although they do not guarantee completely accurate predictions.

6.9 Integration of AI Components

The AI components work together to support inventory decision-making.

The process begins with historical sales and inventory data. The system prepares and processes the data before supplying it to the forecasting model.

The Ridge Regression model estimates product demand. The inventory analysis component then uses the resulting forecasts to assess whether available stock may be sufficient.

The smart restocking component uses the forecast demand and inventory information to calculate suggested order quantities. The dashboard presents the results, while the chatbot gives users another way to request supported information.

The overall process can be summarised as follows:

1. Load historical sales and inventory data.
2. Validate and prepare the dataset.
3. Generate the required forecasting features.
4. Use Ridge Regression to estimate demand.
5. Analyse current inventory against forecast demand.
6. Identify possible stockout risks.

7. Calculate suggested restocking quantities.
8. Display the results through the dashboard.
9. Allow users to access supported information through the chatbot.

This integration connects demand forecasting with practical inventory management decisions.

6.10 Limitations of the AI Implementation

Although the AI components provide useful inventory analysis and forecasting functions, the project has several limitations.

Firstly, the forecasting model uses a prepared retail dataset rather than actual sales records from a participating spaza shop. The results therefore demonstrate the system's functionality using the available dataset and should not be treated as verified predictions for a real shop.

Secondly, the forecasting model depends on the quality and availability of historical data. Limited or inconsistent records may affect prediction accuracy.

Thirdly, the system does not connect to a live point-of-sale system. It does not automatically collect new sales information from a shop's checkout system.

Fourthly, the chatbot uses predefined intent recognition and pattern matching. It cannot respond to every possible question and is limited to the information supported by its implementation.

Finally, the restocking recommendations depend on assumptions about supplier lead time, safety stock and review periods. These values may need adjustment for an actual retailer.

Consider these limitations when interpreting the results and evaluating the system's suitability for real-world use.

6.11 Chapter Summary

This chapter discussed the AI development and implementation of Retail-X-AI. It explained the use of Ridge Regression for demand forecasting, the preparation of historical sales features, model training and evaluation, and the use of time series information.

It also explained how forecast demand supports inventory analysis, stockout risk detection, and smart restocking recommendations. The chapter also described the chatbot's predefined intent recognition approach and how the AI components integrate into the system. Overall, the implemented AI functionality demonstrates how machine learning and data analysis can support inventory management in a small retail environment. However, the results are based on the available prepared dataset and the system's assumptions.

APPLICATION AND DATABASE IMPLEMENTATION

7.1 Introduction

This chapter discusses the implementation of the Retail-X-AI application and its database components. The application was developed using Python and Streamlit to provide users with access to sales analysis, demand forecasting, inventory alerts, restocking recommendations and chatbot functionality.

The project also includes a SQLite database integration service that imports and stores the historical retail dataset. Different application components use prepared CSV files and saved forecasting outputs to display results.

7.2 Application Development

7.2.1 Application Overview

Retail-X-AI was developed as a web-based application using Streamlit. Streamlit lets Python functionality and data analysis results appear in an interactive interface without a separate front-end framework.

The main application is implemented in the root app.py file. It loads the prepared retail dataset and supporting output files, processes the information and displays the results through different application pages.

The application provides store selection and interactive data visualisations to help users understand historical sales and inventory information.

Main application file:

app.py

7.2.2 Application Navigation and Features

The application contains seven navigation pages. Six pages have been opened and confirmed to work during the project demonstration and testing process. The Model Performance page is included in the application, but its operation has not yet been independently confirmed.

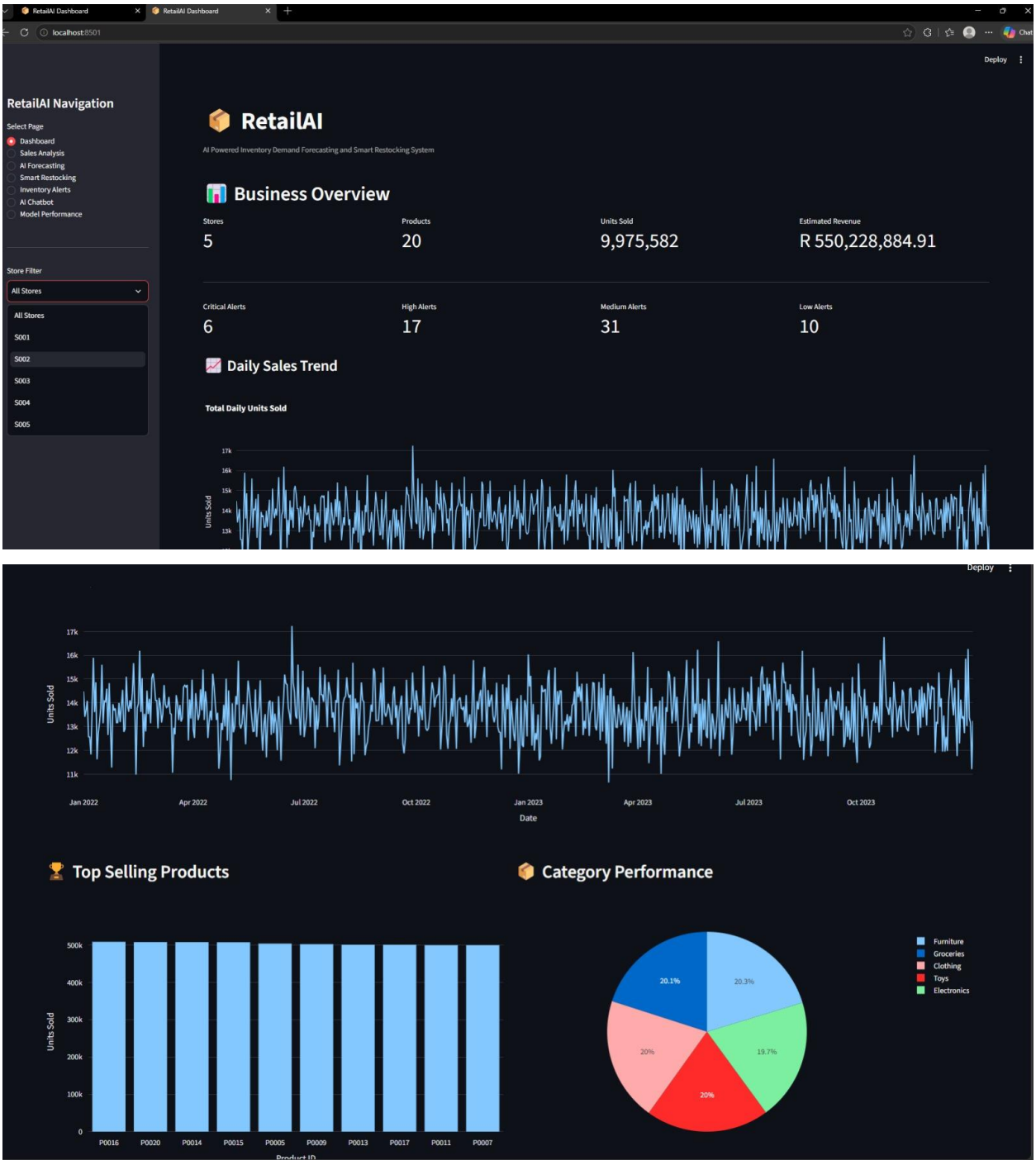
The pages are described below.

1. Dashboard

The Dashboard provides an overview of the available retail information. It displays summary information and visualisations that help users understand overall business activity.

The dashboard includes:

- Total sales information.
- Inventory and sales summaries.
- Inventory alert summaries.
- Daily sales trends.
- Top-selling products.
- Category performance.



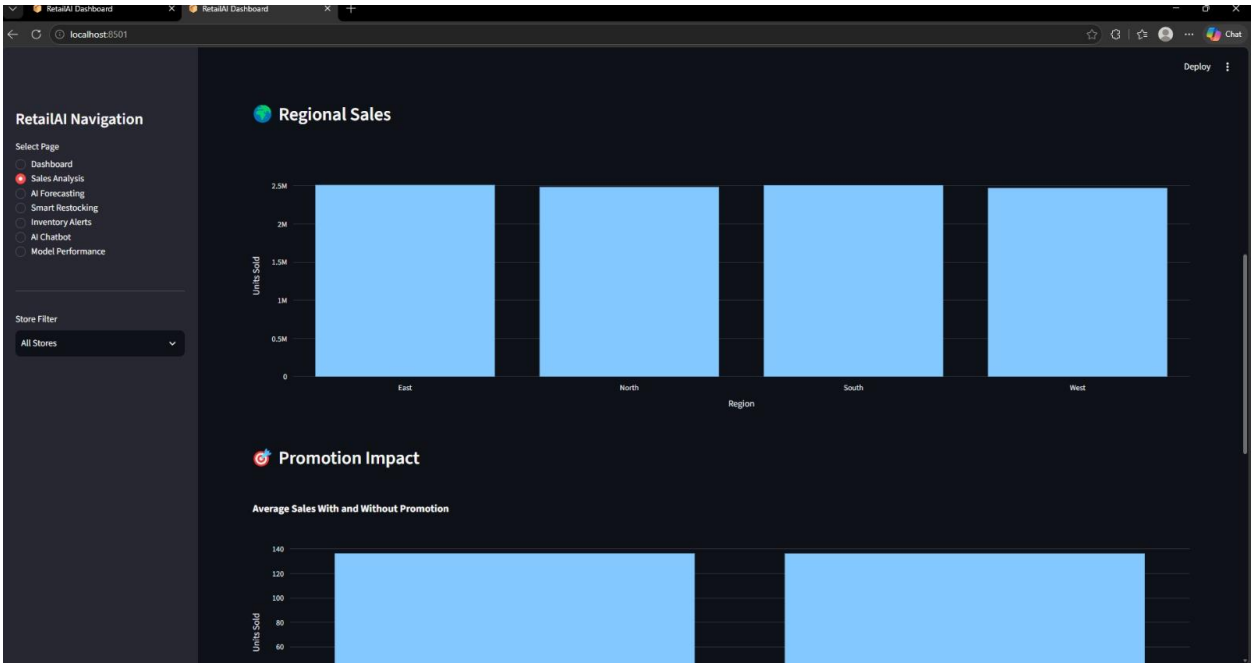
2. Sales Analysis

The Sales Analysis page allows users to examine historical sales information. Users can select a product and view sales summaries and visualisations.

The page includes:

- Total units sold.
- Average units sold.
- Average inventory levels.
- Daily sales trends.
- Regional sales comparisons.
- Promotion impact analysis.



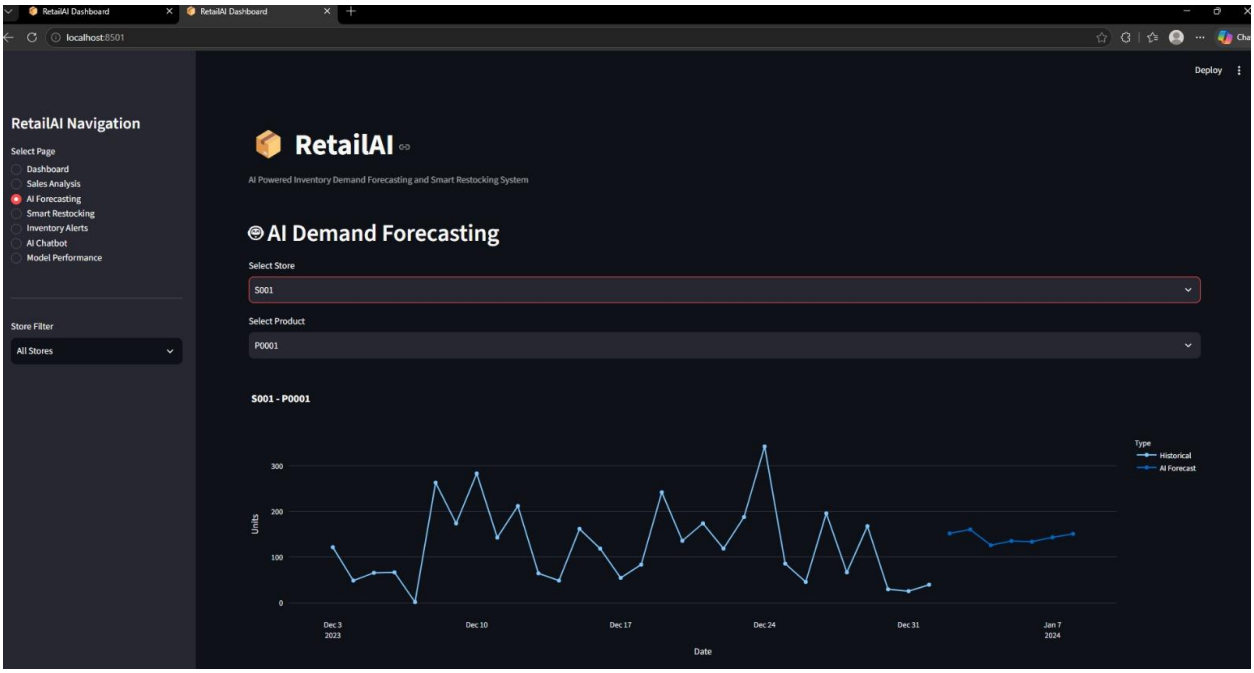


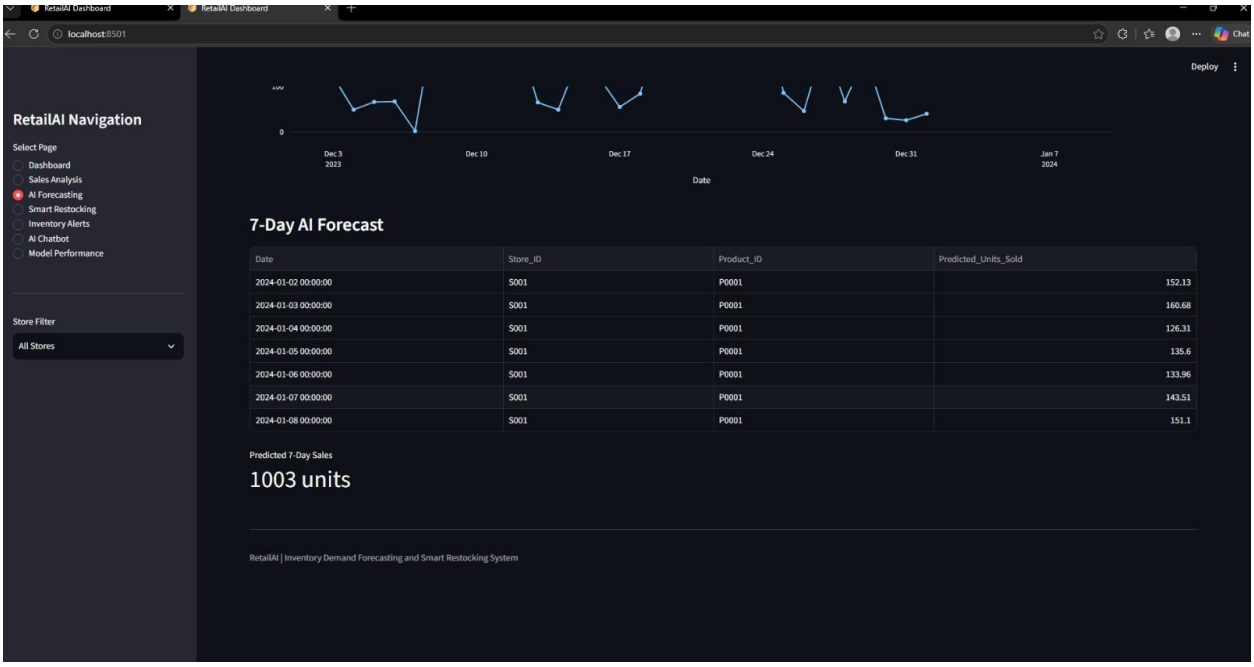
3. AI Forecasting

The AI Forecasting page presents the saved demand forecasting results generated by the selected Ridge Regression model.

Users can examine historical actual sales against predicted sales and view forecasting information using the available filters and visualisations.

The page helps users understand how the model's predictions compare with actual sales records.



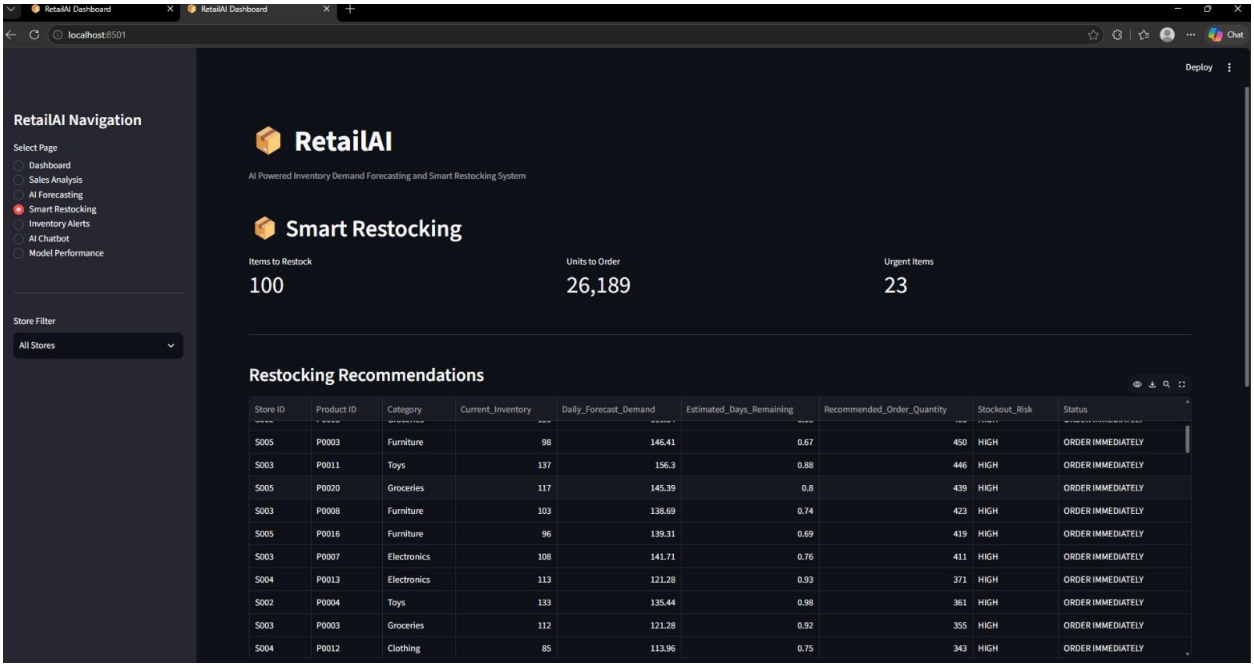


4. Smart Restocking

The Smart Restocking page presents suggested replenishment information based on available inventory and forecast demand.

It helps users identify products that may require attention and view suggested order quantities.

The recommendations are advisory and use historical inventory and forecasting information. They do not automatically place orders with suppliers.



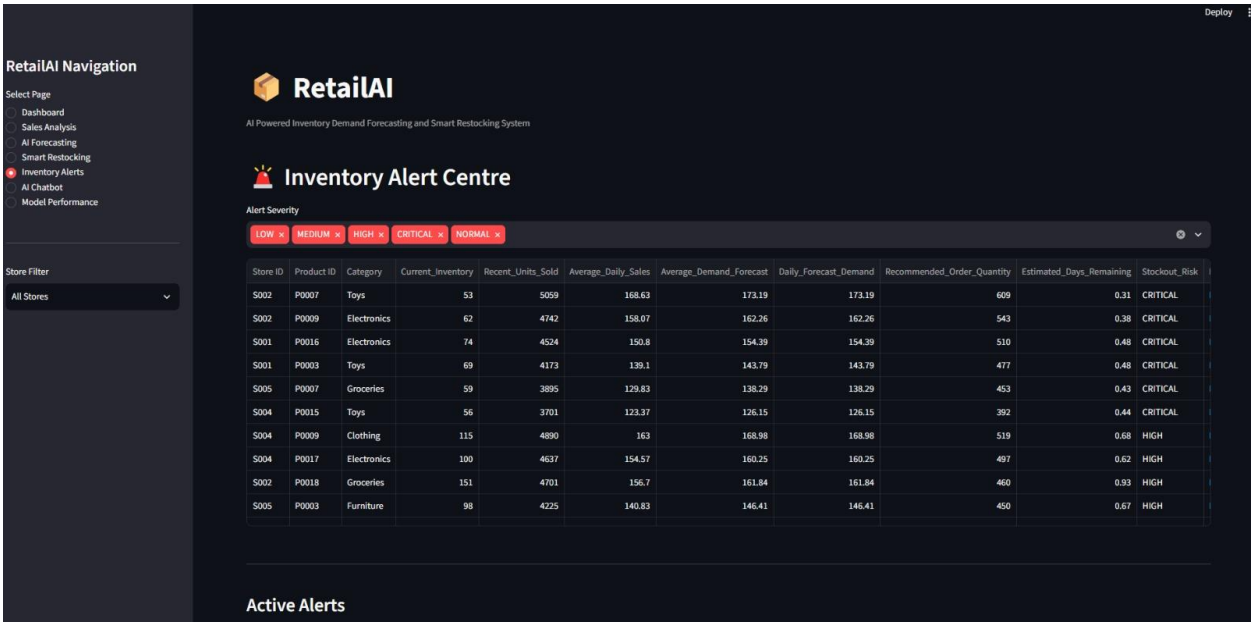
Store ID	Product ID	Category	Current_Inventory	Daily_Forecast_Demand	Estimated_Days_Remaining	Recommended_Order_Quantity	Stockout_Risk	Status
S005	P0003	Furniture	98	146.41	0.67	450	HIGH	ORDER IMMEDIATELY
S003	P0011	Toys	137	156.3	0.88	446	HIGH	ORDER IMMEDIATELY
S005	P0020	Groceries	117	145.39	0.8	439	HIGH	ORDER IMMEDIATELY
S003	P0008	Furniture	103	138.69	0.74	423	HIGH	ORDER IMMEDIATELY
S005	P0016	Furniture	96	139.31	0.69	419	HIGH	ORDER IMMEDIATELY
S003	P0007	Electronics	108	141.71	0.76	411	HIGH	ORDER IMMEDIATELY
S004	P0013	Electronics	113	121.28	0.93	371	HIGH	ORDER IMMEDIATELY
S002	P0004	Toys	133	135.44	0.98	361	HIGH	ORDER IMMEDIATELY
S003	P0003	Groceries	112	121.28	0.92	355	HIGH	ORDER IMMEDIATELY
S004	P0012	Clothing	85	113.96	0.75	343	HIGH	ORDER IMMEDIATELY



5. Inventory Alerts

The Inventory Alerts page displays inventory warnings and helps users identify products that may require attention.

The page presents available alert information, including alert severity, to help users identify more urgent inventory concerns.



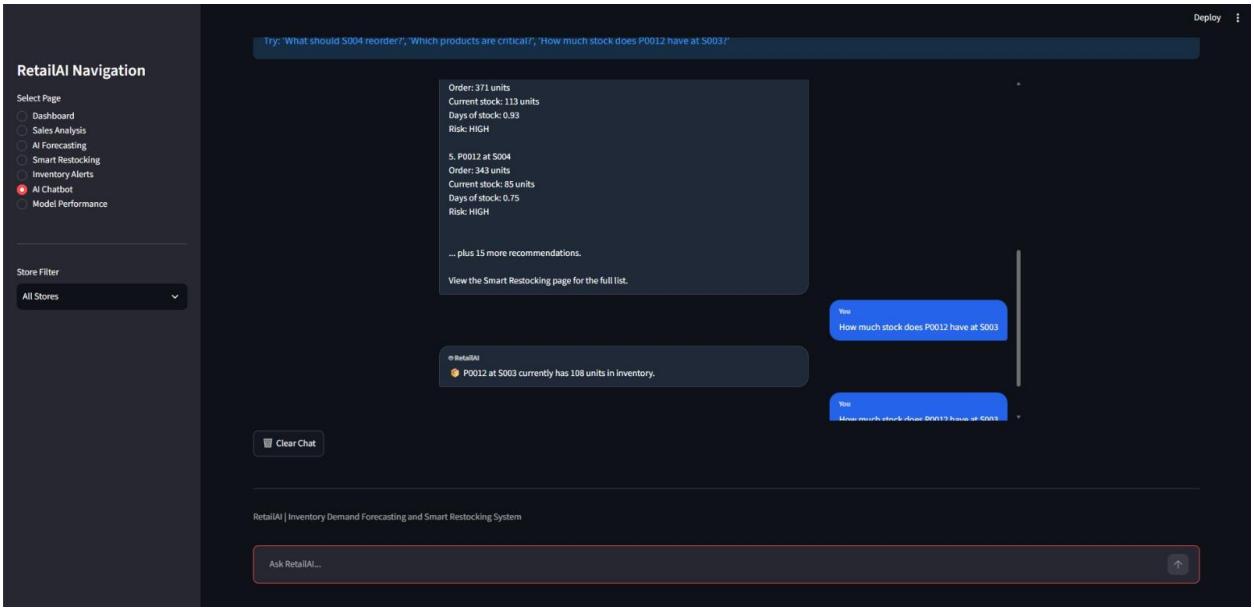


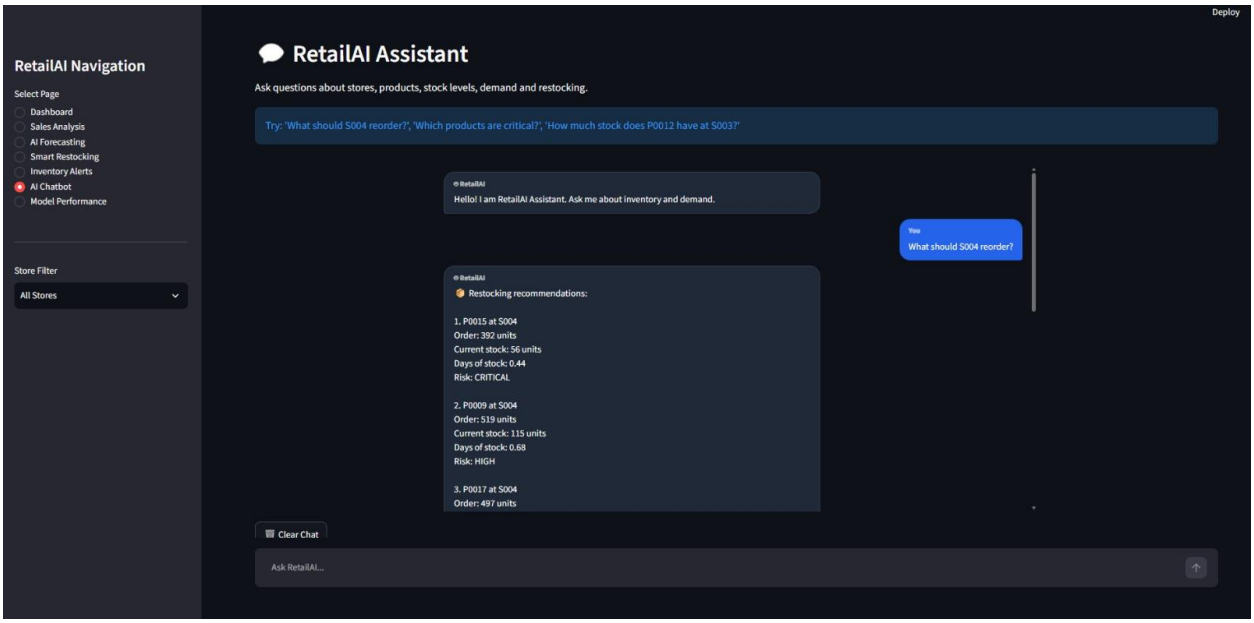
6. AI Chatbot

The AI Chatbot page lets users submit questions about available retail information.

The chatbot uses predefined intent recognition and data queries to identify supported questions and retrieve relevant information. It can assist with questions about sales, inventory, forecasts and restocking recommendations.

It does not use an external large language model or access live shop information.

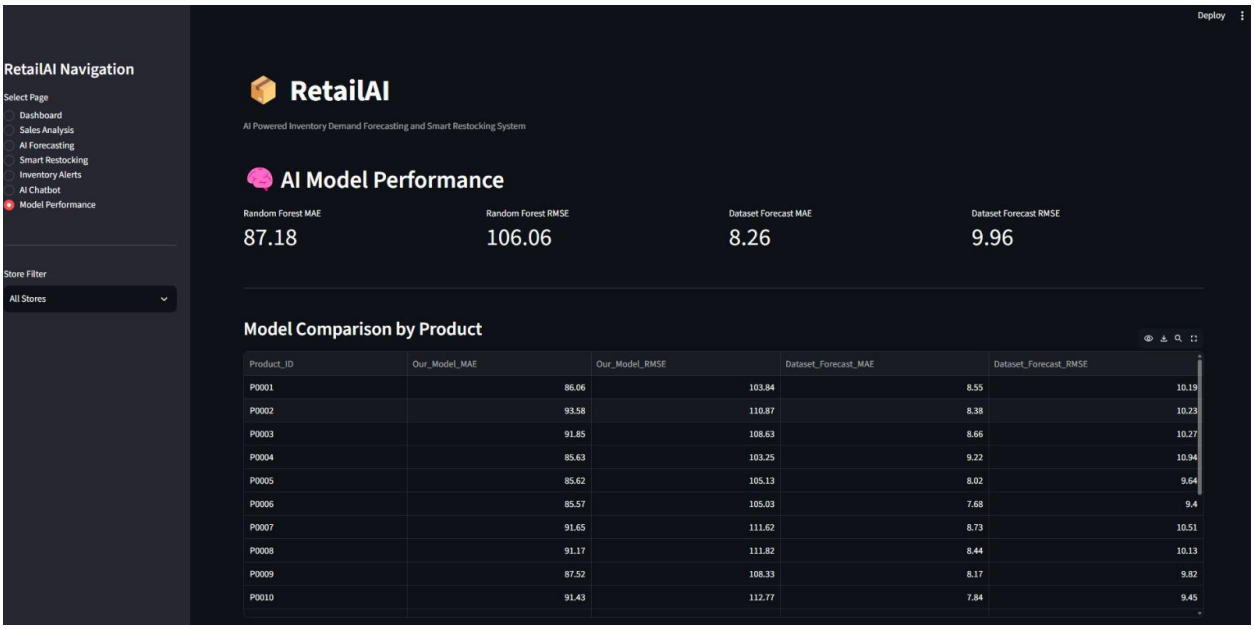




7. Model Performance

The application also contains a Model Performance page intended to present information about forecasting model evaluation.

This page is included in the application navigation. However, its complete functionality has not yet been confirmed, so it should not be presented as a fully tested feature.



.



7.3 Application Structure

The application is organised into different folders to separate the user interface, backend services, database components, AI functionality and supporting documentation.

The main project structure includes:

Folder or File	Purpose
app.py	Main Streamlit application.
application/frontend/	Frontend files for the separate chatbot browser interface.
application/backend/	Backend services for database operations, forecasts, recommendations and chatbot integration.
database/	Database schemas, queries, seed data and database documentation.
dataset/	Raw and processed retail datasets.
python-ai/	Python data processing, forecasting and AI components.
testing/	Automated tests and testing evidence.
diagrams/	System diagrams and design documentation.
documentation/	Project documentation and technical specifications.

This structure separates the project's main responsibilities and makes components easier to locate and maintain.

7.4 Database Implementation

7.4.1 Database Overview

Retail-X-AI includes a SQLite database integration service to store and retrieve historical retail inventory records.

The database service is implemented in:

application/backend/database/service.py

The SQLite database file is generated locally at:

database/retail_inventory.db

The database uses the schema defined in:

database/sqlite_schema.sql

The database service reads the historical retail CSV dataset, checks the required columns and validates important data values before importing the records.

The current dataset contains 73,100 records.

7.4.2 Database Technology

SQLite was used for the implemented database integration service because it provides a local database accessible through Python without requiring a separate database server.

The project also contains a PostgreSQL relational database schema in database/schema.sql. This represents a more structured relational design for the project.

It is important to distinguish the two:

- The implemented Python database service uses SQLite.
- A separate SQL schema represents PostgreSQL.
- The project has not been verified as using a live MySQL database.

7.4.3 Database Table Structure

The implemented SQLite database uses an inventory_records table to store the imported retail dataset.

The table contains the following fields:

Field	Description
record_id	Unique identifier for each database record.

record_date	Date associated with the retail record.
store_id	Identifier of the store.
product_id	Identifier of the product.
category	Product category.
region	Store region.
inventory_level	Available inventory quantity.
units_sold	Number of units sold.
units_ordered	Number of units ordered.
demand_forecast	Demand forecast value included in the source dataset.
price	Product selling price.
discount	Discount applied to the product.
weather_condition	Weather condition recorded for the date.
holiday_promotion	Indicates whether a holiday or promotion was recorded.
competitor_pricing	Competitor pricing information.
seasonality	Seasonal information associated with the record.

The database also contains constraints to prevent invalid inventory quantities, negative prices and discounts outside the permitted range.

Indexes are created on important fields such as date, store, product and category to support data retrieval.

7.4.4 Database Import Process

The database service imports information from:

dataset/raw/retail_store_inventory.csv

The import process follows these steps:

1. The service checks whether the source CSV file exists.
2. It calculates a SHA256 checksum to identify the dataset version.
3. It initialises the SQLite database schema.
4. It checks whether the required dataset columns are available.
5. It renames the dataset columns to match the database field names.
6. It validates missing values, dates, numeric values and important business rules.
7. It imports the records into the inventory_records table.
8. It checks that the number of imported records matches the source dataset.
9. It saves import information to prevent unnecessary re-importing of an unchanged dataset.

This process helps maintain consistency between the source dataset and the database records.

7.4.5 Database Relationships and Logical Design

The project also includes a relational database design consisting of six main entities:

Entity	**Purpose**
CATEGORY	Stores product category information.
REGION	Stores regional information.
STORE	Stores information about retail stores.
PRODUCT	Stores product information.
DATE_DIM	Stores calendar and date information.
INVENTORY_RECORD	Stores daily inventory and sales information.

The logical design separates stores, products, categories, regions and dates from the inventory records.

The PostgreSQL schema defines primary keys and foreign key relationships to connect these entities. This design supports a more structured relational database implementation.

The SQLite implementation currently uses a single inventory records table rather than implementing all six entities separately.

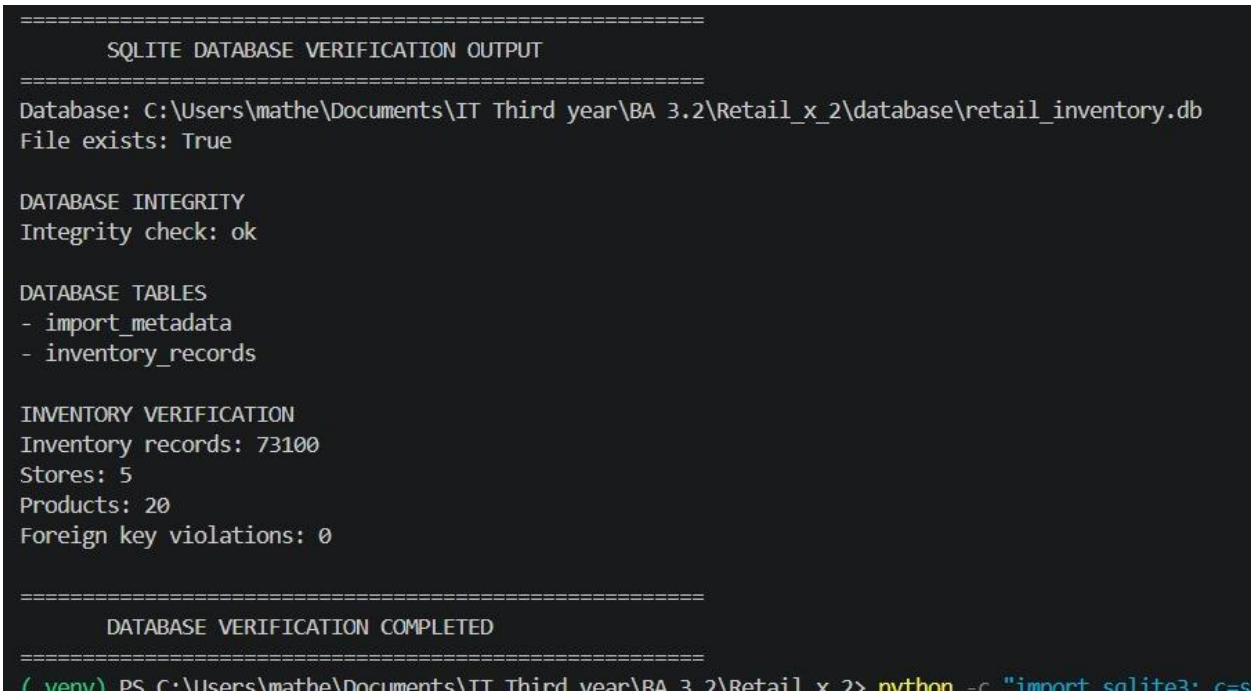
7.4.6 Database Verification

The database service includes a verification process to check whether the imported records are available.

The database can be checked using the following command from the project root directory:

```
python -c "import sqlite3; c=sqlite3.connect('database/retail_inventory.db'); print('Integrity:', c.execute('PRAGMA integrity_check').fetchone()[0]); print('Records:', c.execute('SELECT COUNT(*) FROM inventory_records').fetchone()[0]); c.close()"
```

The project documentation provides the database verification command. Include the actual output as evidence if you perform the verification.



7.5 Integration Between the Application and Database

The project contains backend services that connect the database, forecasting results, inventory analysis and chatbot functionality.

The database service provides functions to access imported inventory records and retrieve inventory summaries.

The forecasting service loads saved forecasting output and provides functions to validate, filter, and summarise forecast records.

The recommendation service combines saved forecasting results with historical inventory records retrieved from SQLite. It then uses the stockout and reorder components to generate advisory recommendations.

The chatbot service connects the application backend to the existing chatbot functionality implemented in the Python AI folder.

Although these services provide integration between different project components, the main Streamlit application also loads prepared CSV files directly for several of its pages.

The project therefore uses both CSV-based application data and a separate SQLite database integration service.

7.6 Application Limitations

The following limitations apply to the current implementation:

1. The application uses historical retail information rather than live point-of-sale transactions.
2. The system does not automatically update stock after a sale.
3. The application does not automatically place orders with suppliers.
4. The SQLite database is generated locally and is not a shared cloud database.
5. The chatbot is limited to predefined intents and supported data queries.
6. The PostgreSQL schema is available as a database design, but the implemented database service uses SQLite.
7. The Model Performance page requires further verification.
8. The system has not been verified through deployment and operation in an actual spaza shop.

These limitations provide opportunities for future development and improvement.

7.7 Summary

This chapter described the implementation of the Retail-X-AI application and its database components. The application was developed using Streamlit and provides pages for dashboard reporting, sales analysis, demand forecasting, smart restocking, inventory alerts and chatbot interaction.

The current implementation provides a working prototype for historical retail inventory analysis. Further development is required to support live point-of-sale integration, automatic inventory updates, and real-world deployment.

TESTING AND RESULTS

8.1 Introduction

This chapter presents the testing activities conducted to evaluate the functionality, performance and reliability of Retail-X-AI. Testing confirmed that the application and its AI components work as expected, identified possible errors, and assessed the quality of the generated results.

The main testing areas included functional testing, AI validation, integration testing, performance testing and security testing.

8.2 Functional Testing

Functional testing verified that the main application features work correctly. Automated testing was performed using Streamlit AppTest in a Windows environment.

The following seven application pages were tested:

1. Dashboard
2. Sales Analysis
3. AI Forecasting
4. Smart Restocking
5. Inventory Alerts
6. AI Chatbot
7. Model Performance

All seven pages executed without application exceptions.

Measurement	**Result**
Automated tests executed	2

Tests passed	2
Tests failed	0
Execution time	6.425 seconds

The results indicate that the application starts successfully and that the tested pages execute without errors.

8.3 AI Forecasting and Restocking Validation

AI validation checked the saved forecasting results and the outputs generated by the inventory and restocking components.

The forecasting validation confirmed that the output contained valid numerical values, correct dates and no duplicate date, store and product combinations. The saved evaluation metrics were also checked against the forecast results.

8.3.1 Forecasting Results

The final Ridge Regression model produced the following results:

Evaluation Measure	**Result**
MAE	88.453325
RMSE	107.996727
Test records	11,000
Unique dates	110

The validation confirmed that the calculated MAE and RMSE matched the saved evaluation results.

8.3.2 Restocking and Stockout Results

The restocking validation produced the following results:

Classification	Records
REORDER_NOW	11,000
STOCKOUT_RISK	5000

5000 records were classified as requiring restocking and as having stockout risk.

Although the numerical output checks passed, the identical classifications raised concerns about the inventory assumptions, demand values and stock risk thresholds. These calculations require further investigation before the recommendations can be considered reliable for actual shop operations.

8.4 Integration Testing

Integration testing was performed to check whether the database, forecasting and restocking components could work together.

The dedicated integration tests produced the following results:

Measurement	Result
Tests executed	4
Tests passed	4
Tests failed	0

Additional tests recorded 8 successful chatbot integration tests, with 1 skipped, and 8 successful forecast integration tests.

The skipped chatbot test required a cleaned inventory dataset that was unavailable during testing.

Overall, the results showed that the tested components could access and process the available project data successfully.

8.5 Performance Testing

Performance testing measured the execution time of selected system operations. Each workflow was measured five times.

Workflow	**Average Execution Time**
Complete restocking workflow	1,355.72 ms
Database summary retrieval	73.93 ms
Forecast CSV loading	102.88 ms

The database summary retrieval was the fastest operation, while the complete restocking workflow required more processing time.

All three dedicated performance tests passed. However, the results were obtained in a local testing environment and do not represent performance under heavy workloads or multiple simultaneous users.

8.6 Security Testing

Security testing was documented under the project's testing/security/ directory.

The available testing summary reports successful checks covering authentication, unauthorised access, input validation, database security, password protection and data privacy.

The overall result was recorded as PASS, with zero failed tests reported.

However, the available summary does not provide detailed individual test results. Therefore, the security findings should be understood as reported summary results rather than proof that every possible security vulnerability has been eliminated.

8.7 Summary of Testing Results

Testing Category	Result
Functional testing	2 tests passed
AI validation	Passed with warnings
Integration testing	4 dedicated tests passed
Additional integration testing	16 passed, 1 skipped
Performance testing	3 tests passed
Security testing	PASS reported

The testing results demonstrate that the main application components executed successfully in the tested environment. However, the AI validation identified issues with the restocking and stockout classifications.

Another difference was identified between the reports: M15-03 records 11,000 restocking records, while M15-06 reports 100 restocking recommendations. Investigate this difference against the original output files and test configurations.

8.8 Testing Limitations

The testing process identified several limitations:

- All validated restocking records received the same REORDER_NOW classification.
- All stock risk records received the same STOCKOUT_RISK classification.

- One chatbot integration test was skipped because the required dataset was unavailable.
- Performance testing was limited to a local environment.
- Detailed evidence for individual security tests was not available.
- The system was tested using prepared historical data rather than live transactions from a real spaza shop.

CONCLUSION AND RECOMMENDATIONS

9.1 Project Conclusion

The Retail-X-AI project demonstrated how artificial intelligence and data analysis can support inventory management and improve stock-related decision-making in small retail environments.

The system was developed in Python using Streamlit, with Ridge Regression as the main demand forecasting model. It analyses historical retail data to estimate product demand and provides inventory risk information and suggested restocking quantities.

The application includes a dashboard, sales analysis, AI forecasting, smart restocking, inventory alerts and an AI chatbot. These features allow users to view sales information, examine demand forecasts, identify potential stock risks and interact with inventory information through predefined chatbot commands.

The forecasting model achieved a Mean Absolute Error (MAE) of 88.453325 and a Root Mean Squared Error (RMSE) of 107.996727 on the final test dataset containing 11,000 records. The results show that the model trained and evaluated successfully, although further improvements are needed to increase forecasting accuracy.

Functional testing also confirmed that the seven application pages ran without exceptions. Additional integration and performance tests assessed the system's main components.

Overall, the project achieved its main technical objective of developing a working AI-based inventory management prototype. However, the system has not yet been tested using actual transactions from a real spaza shop. Its business impact therefore remains to be established through practical implementation.

9.2 Recommendations

1. **Use real business data:** Test the system with actual sales and inventory records from spaza shops to determine whether its forecasts reflect real retail demand.
2. **Improve forecasting accuracy:** Conduct additional feature testing, parameter tuning, and comparisons with other forecasting approaches to improve prediction performance.
3. **Review inventory risk calculations:** Review stockout risk and restocking classifications to ensure recommendations fit different products and inventory conditions.
4. **Introduce live database integration:** The application should be connected to a central database that can store and update sales and inventory information instead of relying mainly on prepared CSV files.
5. **Improve chatbot capabilities:** Expand the chatbot to understand a wider range of questions and provide more detailed responses about sales, stock levels, and restocking recommendations.
6. **Add expiry date monitoring:** Include product expiry dates in the inventory data to help shop owners identify products approaching their expiry dates.
7. **Conduct real-world user testing:** Shop owners and employees should test the application and provide feedback on its usability, accuracy and usefulness.

9.3 Future Improvements

- Integration with point-of-sale systems to capture sales automatically.
- Supplier information and purchase order management.
- Product expiry tracking and notifications.
- Improved chatbot functionality.
- Support for multiple shops and branches.
- User authentication and role-based access control.
- Automated reports showing sales performance and inventory trends.
- Continuous model evaluation using newly collected sales data.

9.4 Final Conclusion

Retail-X-AI provided an opportunity to apply artificial intelligence, machine learning, data preparation and software development techniques to a practical business problem.

The project showed how historical sales and inventory data can generate demand forecasts and support stock management decisions. Although the prototype has achieved its main development objectives, its limitations show that further testing, real-world data integration and improvements to the forecasting and restocking components are necessary.

In conclusion, Retail-X-AI demonstrates the potential of AI-based inventory management systems to support small retailers by providing better access to sales information, demand estimates and stock-related recommendations. With further development and real-world testing, the system could become a useful tool for helping spaza shop owners make more informed inventory decisions.

10. DECLARATION OF OWN WORK

Wemembers of this group, hereby declare that this project documentation and all accompanying materials are our own original work. All sources of information used in the preparation of this document have been duly acknowledged and referenced. We understand that plagiarism is a serious academic offence that may result in expulsion from the University. We further confirm that we have not copied the work of any other group or individual, and that each member has contributed meaningfully to the completion of this project.

We also confirm that a Grammarly report (or equivalent proof of language checking) has been included as an annexure to this submission, and that the document has been prepared using Arial font, size 12, 1.5-line spacing, and justified text alignment as required.

Signatures of all group members:

Name and surname	signature
KT Mudau	
TSC Mathelela	
K Dlamini	
M Mapheto	





11.Grammarly report

Report: Business Analysis Project Overview

Business Analysis Project Overview

by Nyambeni Mudau

General metrics

76,009	10,153	995	40 min 36 sec	1 hr 18 min
characters	words	sentences	reading time	speaking time

Writing Issues

No issues found

Plagiarism

This text hasn't been checked for plagiarism

Unique Words

Measures vocabulary diversity by calculating the percentage of words used only once in your document

12%

unique words

Rare Words

Measures depth of vocabulary by identifying words that are not among the 5,000 most common English words.

46%

rare words

Word Length

Measures average word length

6.1

characters per word

Sentence Length

Measures average sentence length

10.2

words per sentence